

Source Code Documentation

Generated: 2026-07-10 15:30

```
$ generate_source_pdfs.py .. -e .py -o stockChecker_source.pdf
```

Contents

stockChecker ¥468¥468¥468¥468¥468¥468¥468¥468¥468¥468

app.py

stockChecker¥app.py

125 lines | 5,081 bytes

```
1 """stockChecker メインエントリーポイント
2
3 Streamlit アプリケーションを起動し、バックテストの実行および結果表示を行う。
4 ユーザーは基準日を選択し「バックテスト実行」ボタンを押すことで、
5 全ルールによるスコアリング結果をタブ形式で確認できる。
6 """
7
8 import streamlit as st
9 from datetime import date
10
11 from db.schema import init_db
12 from rules.runner import discover_rules, run_backtest, get_rule_names
13 from ui.tabs import render_tab, render_total_tab
14 from ui.ranking import build_total_ranking_simple
15
16
17 # Streamlit の set_page_config は他の st 描画命令より先に呼ぶ必要がある（最初の描画命令）
18 st.set_page_config(
19     page_title="バックテストシステム",
20     layout="wide",
21     initial_sidebar_state="expanded",
22 )
23
24
25 def main():
26     """Streamlit UI を構築し、バックテストの実行・結果表示を制御する。
27
28     左カラムに基準日選択、右カラムに実行ボタンを配置し、
29     バックテスト完了後はルールごとのタブと総合ランキングタブを表示する。
30
31     Args:
32         なし
33
34     Returns:
35         None
36
37     Raises:
38         なし (Streamlit の内部エラーは st.error でハンドルされない)
39     """
40     init_db()
41
42     st.title("資産運用 バックテストシステム")
43     st.markdown("データマイニングで発見した「法則」を過去データで検証します")
44
45     rules = discover_rules()
46     rule_names = get_rule_names()
47
48     st.markdown(
49         "<style>"
50         "div[data-baseweb='popover'] {"
51         "  transform: translateX(18.00vw) translateY(180px) !important;"
52         "}"
53         "</style>",
54         unsafe_allow_html=True,
55     )
56
57     col1, col2, col3 = st.columns([1, 3, 2])
58
59     with col1:
60         base_date = st.date_input(
61             "基準日",
62             value=date.today(),
63             max_value=date.today(),
64         )
65
66     with col3:
67         run_button = st.button("バックテスト実行", type="primary")
68
69     # Streamlit の処理モデル: ボタンが押されるとスクリプト全体が上から再実行される
70     if run_button:
71         base_date_str = base_date.strftime("%Y-%m-%d")
```

```

72     with st.spinner("バックテスト実行中..."):
73         results = run_backtest(base_date_str)
74         st.session_state["results"] = results
75         st.session_state["base_date"] = base_date_str
76         st.success(f"バックテスト完了！ 基準日: {base_date_str}")
77
78 # st.session_state は Streamlit のセッション変数。再実行後も値が保持される
79 if "results" in st.session_state:
80     results = st.session_state["results"]
81     base_date_str = st.session_state["base_date"]
82
83     tab_names = ["総合ランキング"] + [r.name for r in rules]
84     tabs = st.tabs(tab_names)
85
86     total_results = build_total_ranking_simple(results)
87
88     with tabs[0]:
89         if total_results:
90             render_total_tab(total_results, base_date_str, results)
91         else:
92             st.info("総合ランキングを計算するにはバックテストを実行してください")
93
94 # enumerate はリストの要素とインデックスを同時に取得する組み込み関数
95 for i, rule in enumerate(rules):
96     with tabs[i + 1]:
97         rule_name = rule.name
98         if rule_name in results:
99             render_tab(rule_name, results[rule_name], base_date_str)
100     else:
101         st.info(f"「{rule_name}」の結果がありません")
102
103 else:
104     st.info("基準日を選択して「バックテスト実行」ボタンを押してください ")
105
106     st.markdown("""
107     ### 組み込みの法則一覧
108     | 法則 | 説明 |
109     |-----|-----|
110     | モメンタム | 過去3~12ヶ月で上昇した銘柄はその後もし上昇しやすい |
111     | バリュースコア | 割安株 (PBR・PERが低い) は長期的に市場平均を上回りやすい |
112     | クオリティ | 質の良い企業 (ROE高・自己資本率高) は安定したリターンを生む |
113     | サイズ | 小型株は大型株より長期的に高いリターンを出しやすい |
114     | 低ボラティリティ | 値動きが小さい株のほうが長期的にリターンが高い |
115     | PEAD | 良い決算を出した企業は発表後も上昇しやすい |
116     | センチメント | ポジティブニュースが増えると株価が上がりやすい |
117     | テキスト特徴 | 決算書の楽観的な文章は株価上昇と相関する |
118     | 異常検知 | 出来高・ボラティリティ急増銘柄は大きく動きやすい |
119     """)
120
121
122 # __name__ == "__main__" は「このファイルが直接実行されたときだけ実行する」という Python のイディオム
123 # import されたときは __name__ がファイル名になるので main() は呼ばれない
124 if __name__ == "__main__":
125     main()

```

config.py

stockChecker¥config.py

36 lines | 1,712 bytes

```
1 """stockChecker 設定モジュール
2
3 環境変数 (.env ファイル) から各種設定値を読み込み、
4 データベースパス・API キー・ダウンロード期間などをモジュール定数として提供する。
5 """
6
7 import os
8 from dotenv import load_dotenv
9
10 # .env ファイルの内容を os.environ に読み込む (環境変数として扱えるようになる)
11 load_dotenv()
12
13 # os.getenv(key, default) は環境変数が未定義の場合にデフォルト値を返す (安全な読み取り)
14 DB_PATH = os.getenv("DB_PATH", "backtest.db")
15
16 # int() で文字列→整数に変換 (.env の値はすべて文字列として読まれるため)
17 SENTIMENT_BATCH_SIZE = int(os.getenv("SENTIMENT_BATCH_SIZE", "50"))
18 SENTIMENT_MIN_INTERVAL_HOURS = int(os.getenv("SENTIMENT_MIN_INTERVAL_HOURS", "24"))
19 MAX_RETRIES = int(os.getenv("MAX_RETRIES", "3"))
20 RETRY_WAIT_SECONDS = int(os.getenv("RETRY_WAIT_SECONDS", "60"))
21 USER_AGENT = os.getenv("USER_AGENT", "BacktestSystem/1.0")
22
23 NEWSAPI_KEY = os.getenv("NEWSAPI_KEY", "")
24 FINNHUB_API_KEY = os.getenv("FINNHUB_API_KEY", "")
25 EDINET_API_KEY = os.getenv("EDINET_API_KEY", "")
26
27 FINBERT_MODEL_NAME = os.getenv("FINBERT_MODEL_NAME", "ProsusAI/finbert")
28
29 PRICE_LOOKBACK_YEARS = int(os.getenv("PRICE_LOOKBACK_YEARS", "10"))
30 FINANCIAL_LOOKBACK_YEARS = int(os.getenv("FINANCIAL_LOOKBACK_YEARS", "4"))
31
32 JPX_TICKER_URLS = {
33     "prime": "https://www.jpx.co.jp/markets/statistics-equities/misc/tvdivq0000001vg2-att/listed_co_j.csv",
34     "standard": "https://www.jpx.co.jp/markets/statistics-equities/misc/tvdivq0000001vg2-att/listed_co_s.csv",
35     "growth": "https://www.jpx.co.jp/markets/statistics-equities/misc/tvdivq0000001vg2-att/listed_co_g.csv",
36 }
```

__init__.py

stockChecker¥db¥__init__.py

1 lines | 0 bytes

1

downloader_financials.py

stockChecker¥db¥downloader_financials.py

284 lines | 10,760 bytes

```
1  """財務データのダウンロード・保存
2
3  yfinance または EDINET から財務諸表データを取得し、
4  financials テーブルへの保存を行う。
5  """
6
7  import time
8  import yfinance as yf
9  import pandas as pd
10 from datetime import datetime
11 from typing import Optional
12
13 from config import FINANCIAL_LOOKBACK_YEARS, MAX_RETRIES, RETRY_WAIT_SECONDS
14 from db.schema import get_connection, log_error
15
16
17 # yfinance の ticker.info は 1 回の API 呼び出しで多数のファンダメンタル指標を取得できる
18 # ticker.financials / balance_sheet / cashflow は縦軸が勘定科目、横軸が年度の DataFrame
19 def fetch_yfinance_financials(symbol: str) -> Optional[dict]:
20     """指定銘柄の財務データを yfinance から取得する。
21
22     info (PER・PBR・ROE・時価総額等) と financials / balance_sheet / cashflow の
23     各 DataFrame をまとめて返す。
24
25     Args:
26         symbol: 銘柄シンボル (例: "AAPL")。
27
28     Returns:
29         Optional[dict]: info と各 DataFrame を含む辞書。失敗時は None。
30
31     Raises:
32         なし (例外は関数内で捕捉され、リトライ後に None を返す)。
33     """
34     for attempt in range(MAX_RETRIES):
35         try:
36             ticker = yf.Ticker(symbol)
37             info = ticker.info
38             result = {
39                 "per": info.get("trailingPE"),
40                 "pbr": info.get("priceToBook"),
41                 "roe": info.get("returnOnEquity"),
42                 "dividend_yield": info.get("dividendYield"),
43                 "market_cap": info.get("marketCap"),
44                 "revenue": info.get("totalRevenue"),
45                 "operating_margin": info.get("operatingMargins"),
46             }
47
48             financials = ticker.financials
49             balance_sheet = ticker.balance_sheet
50             cashflow = ticker.cashflow
51
52             result["financials_df"] = financials
53             result["balance_sheet_df"] = balance_sheet
54             result["cashflow_df"] = cashflow
55             result["info"] = info
56             return result
57         except Exception as e:
58             if attempt < MAX_RETRIES - 1:
59                 time.sleep(RETRY_WAIT_SECONDS)
60             else:
61                 return None
62
63
64 def extract_fiscal_year_data(raw: dict) -> list:
65     """yfinance の財務 DataFrame から会計年度別のデータを抽出する。
66
67     financials / balance_sheet / cashflow の各 DataFrame を横断し、
68     各会計年度の収益・営業利益・純利益・総資産・自己資本・キャッシュフローを
69     辞書リストとして整列する。
70
71     Args:
```

```

72     raw: fetch_yfinance_financials の戻り値。
73
74 Returns:
75     list: 各要素が fiscal_year / revenue / operating_income / net_income /
76           total_assets / total_equity / cash_flow / per / pbr / roe /
77           dividend_yield を含む辞書のリスト。
78
79 注意:
80     - financials が空の場合、PER・PBR・ROE のみの簡易レコード 1 件を返す。
81     ""
82 records = []
83 financials = raw.get("financials_df")
84 balance_sheet = raw.get("balance_sheet_df")
85 cashflow = raw.get("cashflow_df")
86 info = raw.get("info", {})
87
88 if financials is None or financials.empty:
89     current_year = datetime.now().year
90     records.append({
91         "fiscal_year": current_year - 1,
92         "revenue": raw.get("revenue"),
93         "operating_income": None,
94         "net_income": None,
95         "total_assets": None,
96         "total_equity": None,
97         "cash_flow": None,
98         "per": raw.get("per"),
99         "pbr": raw.get("pbr"),
100        "roe": raw.get("roe"),
101        "dividend_yield": raw.get("dividend_yield"),
102    })
103    return records
104
105 for col in financials.columns:
106     try:
107         year = col.year if hasattr(col, "year") else int(col)
108     except (ValueError, TypeError):
109         continue
110
111     rev = financials.loc["Total Revenue"] if "Total Revenue" in financials.index else None
112     op_inc = financials.loc["Operating Income"] if "Operating Income" in financials.index else None
113     net_inc = financials.loc["Net Income"] if "Net Income" in financials.index else None
114
115     total_assets = None
116     total_equity = None
117     if balance_sheet is not None and not balance_sheet.empty:
118         if "Total Assets" in balance_sheet.index:
119             total_assets = balance_sheet.loc["Total Assets"].get(col)
120         if "Stockholders Equity" in balance_sheet.index:
121             total_equity = balance_sheet.loc["Stockholders Equity"].get(col)
122         elif "Total Equity Gross Minority Interest" in balance_sheet.index:
123             total_equity = balance_sheet.loc["Total Equity Gross Minority Interest"].get(col)
124
125     cf = None
126     if cashflow is not None and not cashflow.empty:
127         if "Operating Cash Flow" in cashflow.index:
128             cf = cashflow.loc["Operating Cash Flow"].get(col)
129         elif "Free Cash Flow" in cashflow.index:
130             cf = cashflow.loc["Free Cash Flow"].get(col)
131
132     records.append({
133         "fiscal_year": year,
134         "revenue": float(rev.get(col)) if rev is not None and col in rev else None,
135         "operating_income": float(op_inc.get(col)) if op_inc is not None and col in op_inc else None,
136         "net_income": float(net_inc.get(col)) if net_inc is not None and col in net_inc else None,
137         "total_assets": float(total_assets) if total_assets is not None else None,
138         "total_equity": float(total_equity) if total_equity is not None else None,
139         "cash_flow": float(cf) if cf is not None else None,
140         "per": raw.get("per") if records else None,
141         "pbr": raw.get("pbr") if records else None,
142         "roe": raw.get("roe") if records else None,
143         "dividend_yield": raw.get("dividend_yield") if records else None,
144     })
145
146 return records
147
148

```



```

149 def fetch_edinet_financials(symbol: str) -> Optional[list]:
150     """EDINET (日本版 EDGAR) から財務データを取得する。
151
152     非同期クライアントを使用して EDINET から有価証券報告書を取得し、
153     財務指標を計算して extract_fiscal_year_data 互換の形式で返す。
154
155     Args:
156         symbol: 銘柄シンボル (".T" サフィックス付き日本株)。
157
158     Returns:
159         Optional[list]: 財務レコードのリスト。失敗時は None。
160
161     注意:
162     - edinet パッケージがインストールされている必要がある。
163     - 現在は最新年度のみデータを返す。
164     """
165     try:
166         from edinet import EdinetClient
167         import asyncio
168
169         edinet_code = symbol.replace(".T", "")
170
171         async def fetch():
172             async with EdinetClient() as client:
173                 companies = await client.search_companies(edinet_code)
174                 if not companies:
175                     return None
176                 stmt = await client.get_financial_statements(
177                     companies[0].edinet_code
178                 )
179                 if stmt is None:
180                     return None
181                 metrics = client.calculate_metrics(stmt)
182                 return [
183                     "fiscal_year": datetime.now().year - 1,
184                     "revenue": stmt.income_statement.get("売上高"),
185                     "operating_income": stmt.income_statement.get("営業利益"),
186                     "net_income": stmt.income_statement.get("当期純利益"),
187                     "total_assets": stmt.balance_sheet.get("総資産"),
188                     "total_equity": stmt.balance_sheet.get("純資産"),
189                     "cash_flow": None,
190                     "per": None,
191                     "pbr": None,
192                     "roe": metrics.get("profitability", {}).get("ROE"),
193                     "dividend_yield": None,
194                 ]
195
196             return asyncio.run(fetch())
197     except Exception as e:
198         print(f"edinet fetch failed for {symbol}: {e}")
199         return None
200
201
202 def download_all() -> None:
203     """全アクティブ銘柄の財務データを一括ダウンロードする。
204
205     日本株は EDINET、US 株は yfinance を使用して取得する。
206     DB への保存後、各銘柄間にスリープを入れて API 負荷を軽減する。
207
208     Returns:
209         None
210
211     注意:
212     - ダウンロード失敗銘柄は error_log に記録される。
213     """
214     conn = get_connection()
215     cursor = conn.cursor()
216     cursor.execute("SELECT id, symbol, market FROM tickers WHERE is_active = 1")
217     tickers = cursor.fetchall()
218     conn.close()
219
220     total = len(tickers)
221     for idx, row in enumerate(tickers):
222         ticker_id = row["id"]
223         symbol = row["symbol"]
224         market = row["market"]
225         print(f"[{idx+1}/{total}] Fetching financials: {symbol}")

```

```

226
227     records = None
228     if market == "japan":
229         records = fetch_edinet_financials(symbol)
230
231     if records is None:
232         raw = fetch_yfinance_financials(symbol)
233         if raw is not None:
234             records = extract_fiscal_year_data(raw)
235
236     if records:
237         save_financials(ticker_id, records)
238     else:
239         log_error("download_financials", ticker_id, f"No financial data for {symbol}")
240
241     time.sleep(0.3 if market == "us" else 0.5)
242
243     print("Financial data download complete.")
244
245
246 def update_accumulate() -> None:
247     """財務データの一括更新 (download_all のラッパー) 。
248
249     download_all() を呼び出し、全アクティブ銘柄の財務データを再取得する。
250     """
251     download_all()
252
253
254 def save_financials(ticker_id: int, records: list) -> None:
255     """財務レコードのリストを financials テーブルに保存する。
256
257     Args:
258         ticker_id: 銘柄 ID。
259         records: extract_fiscal_year_data 互換の辞書リスト。
260
261     Returns:
262         None
263
264     Raises:
265         sqlite3.Error: DB 書き込みに失敗した場合。
266     """
267     conn = get_connection()
268     cursor = conn.cursor()
269
270     for rec in records:
271         cursor.execute("""
272             INSERT OR IGNORE INTO financials
273                 (ticker_id, fiscal_year, revenue, operating_income, net_income,
274                  total_assets, total_equity, cash_flow, per, pbr, roe, dividend_yield)
275             VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
276         """, (
277             ticker_id, rec["fiscal_year"],
278             rec["revenue"], rec["operating_income"], rec["net_income"],
279             rec["total_assets"], rec["total_equity"], rec["cash_flow"],
280             rec["per"], rec["pbr"], rec["roe"], rec["dividend_yield"],
281         ))
282
283     conn.commit()
284     conn.close()

```

downloader_prices.py

stockChecker¥db¥downloader_prices.py

164 lines | 5,380 bytes

```
1  """株価データのダウンロード・保存
2
3  yfinance を使用して指定銘柄の日次株価を取得し、
4  prices テーブルへの保存・差分更新を行う。
5  """
6
7  import time
8  import yfinance as yf
9  import pandas as pd
10 from datetime import datetime, timedelta
11 from typing import Optional
12
13 from config import PRICE_LOOKBACK_YEARS, MAX_RETRIES, RETRY_WAIT_SECONDS
14 from db.schema import get_connection, log_error
15
16
17 # MAX_RETRIES 回のリトライループ - ネットワーク一時障害で落ちても自動再試行する
18 def download_single_price(symbol: str, start: str, end: str) -> Optional[pd.DataFrame]:
19     """指定銘柄の株価履歴を yfinance からダウンロードする。
20
21     リトライ処理付きで、auto_adjust=True (配当・分割調整済み) で取得する。
22
23     Args:
24         symbol: 銘柄シンボル (例: "AAPL")。
25         start: 開始日 ("YYYY-MM-DD" 形式)。
26         end: 終了日 ("YYYY-MM-DD" 形式)。
27
28     Returns:
29         Optional[pd.DataFrame]: yfinance の履歴 DataFrame。失敗時は None。
30
31     Raises:
32         なし (例外は関数内で捕捉され、リトライ後に None を返す)。
33     """
34     for attempt in range(MAX_RETRIES):
35         try:
36             ticker = yf.Ticker(symbol)
37             df = ticker.history(start=start, end=end, auto_adjust=True)
38             if df is not None and not df.empty:
39                 return df
40             return None
41         except Exception as e:
42             if attempt < MAX_RETRIES - 1:
43                 time.sleep(RETRY_WAIT_SECONDS)
44             else:
45                 return None
46
47
48 def download_all() -> None:
49     """全アクティブ銘柄の株価履歴を一括ダウンロードする。
50
51     Returns:
52         None
53
54     注意:
55     - 各銘柄間に 0.2 秒のスリープを入れ、API 負荷を軽減する。
56     - ダウンロード失敗銘柄は error_log に記録される。
57     """
58     conn = get_connection()
59     cursor = conn.cursor()
60     cursor.execute("SELECT id, symbol FROM tickers WHERE is_active = 1")
61     tickers = cursor.fetchall()
62     conn.close()
63
64     end_date = datetime.now().strftime("%Y-%m-%d")
65     start_date = (datetime.now() - timedelta(days=365 * PRICE_LOOKBACK_YEARS)).strftime("%Y-%m-%d")
66
67     total = len(tickers)
68     for idx, row in enumerate(tickers):
69         ticker_id = row["id"]
70         symbol = row["symbol"]
71         print(f"[{idx+1}/{total}] Downloading prices: {symbol}")
```

```

72
73     df = download_single_price(symbol, start_date, end_date)
74     if df is None:
75         log_error("download_price", ticker_id, f"No data for {symbol}")
76         continue
77
78     save_prices(ticker_id, df)
79     time.sleep(0.2)
80
81     print("Price download complete.")
82
83
84 # COALESCE(MAX(p.date), '1900-01-01') - 価格データが 1 件もない銘柄は '1900-01-01' を初期値とする
85 def update_incremental() -> None:
86     """ 最終取得日以降の株価データを差分更新する。
87
88     Returns:
89         None
90
91     注意:
92         - 最終取得日の 5 日前から再取得し、欠損を補完する。
93         - 当日のデータが既に存在する銘柄はスキップされる。
94     """
95     conn = get_connection()
96     cursor = conn.cursor()
97     cursor.execute("""
98         SELECT t.id, t.symbol, COALESCE(MAX(p.date), '1900-01-01') as last_date
99         FROM tickers t
100        LEFT JOIN prices p ON t.id = p.ticker_id
101        WHERE t.is_active = 1
102        GROUP BY t.id, t.symbol
103    """)
104     tickers = cursor.fetchall()
105     conn.close()
106
107     end_date = datetime.now().strftime("%Y-%m-%d")
108
109     for row in tickers:
110         ticker_id = row["id"]
111         symbol = row["symbol"]
112         last_date = row["last_date"]
113
114         if last_date >= end_date:
115             continue
116
117         start_date = (datetime.strptime(last_date, "%Y-%m-%d") - timedelta(days=5)).strftime("%Y-%m-%d")
118         print(f"Updating: {symbol} from {start_date}")
119
120         df = download_single_price(symbol, start_date, end_date)
121         if df is not None and not df.empty:
122             save_prices(ticker_id, df)
123
124         time.sleep(0.2)
125
126     print("Incremental update complete.")
127
128
129 def save_prices(ticker_id: int, df: pd.DataFrame) -> None:
130     """ 株価 DataFrame を prices テーブルに保存する。
131
132     Args:
133         ticker_id: 銘柄 ID。
134         df: yfinance 形式の株価 DataFrame (Open / High / Low / Close / Volume / Dividends / Stock Splits) 。
135
136     Returns:
137         None
138
139     Raises:
140         sqlite3.Error: DB 書き込みに失敗した場合。
141     """
142     conn = get_connection()
143     cursor = conn.cursor()
144
145     rows = []
146     for date_idx, row in df.iterrows():
147         date_str = pd.Timestamp(date_idx).strftime("%Y-%m-%d")
148         rows.append((

```

```

149         ticker_id, date_str,
150         row.get("Open"), row.get("High"),
151         row.get("Low"), row.get("Close"),
152         int(row.get("Volume", 0)) if pd.notna(row.get("Volume")) else 0,
153         float(row.get("Dividends", 0)) if pd.notna(row.get("Dividends")) else 0.0,
154         float(row.get("Stock Splits", 1)) if pd.notna(row.get("Stock Splits")) else 1.0,
155     ))
156
157     cursor.executemany("""
158         INSERT OR REPLACE INTO prices
159         (ticker_id, date, open, high, low, close, volume, dividends, splits)
160         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
161     """, rows)
162
163     conn.commit()
164     conn.close()

```

downloader_sentiment.py

stockChecker¥db¥downloader_sentiment.py

286 lines | 9,376 bytes

```
1  """ニュースセンチメントのダウンロード・分析モジュール
2
3  NewsAPI / Finnhub からニュースを取得し、FinBERTで感情分析を行い、
4  結果を indicators テーブルに保存する。
5  """
6
7  import time
8  import json
9  import requests
10 from datetime import datetime, timedelta
11 from typing import Optional
12
13 from config import (
14     SENTIMENT_BATCH_SIZE, SENTIMENT_MIN_INTERVAL_HOURS,
15     MAX_RETRIES, RETRY_WAIT_SECONDS,
16     NEWSAPI_KEY, FINNHUB_API_KEY, FINBERT_MODEL_NAME,
17 )
18 from db.schema import get_connection
19
20
21 # モジュールレベルの変数 _finbert_pipeline を None で初期化 → 初回呼び出し時に pipeline を生成
22 # 2 回目以降は None でないため、if ブロックを通らずキャッシュされたインスタンスを返す
23 # このパターンを「シングルトン」と呼ぶ（処理の重い初期化を 1 回だけ実行）
24 _finbert_pipeline = None
25
26
27 def get_finbert():
28     """FinBERT 感情分析パイプラインをシングルトンで取得する。
29
30     初回呼び出し時に transformers.pipeline を初期化し、
31     以降はキャッシュされたインスタンスを返す。
32
33     Returns:
34         pipeline: Hugging Face の sentiment-analysis パイプライン。
35     """
36     global _finbert_pipeline
37     if _finbert_pipeline is None:
38         from transformers import pipeline
39         _finbert_pipeline = pipeline(
40             "sentiment-analysis",
41             model=FINBERT_MODEL_NAME,
42             tokenizer=FINBERT_MODEL_NAME,
43         )
44     return _finbert_pipeline
45
46
47 def analyze_sentiment(texts: list[str]) -> list[dict]:
48     """テキストリストに対して FinBERT で感情分析を実行する。
49
50     バッチ処理（32 件単位）で推論を行い、各テキストの感情ラベルと信頼度スコアを返す。
51
52     Args:
53         texts: 分析対象のテキストリスト。
54
55     Returns:
56         list[dict]: 各要素が {"label": str, "score": float} のリスト。
57                     推論失敗要素は {"label": "neutral", "score": 0.5} で補完される。
58
59     注意:
60     - バッチ間には 0.1 秒のスリープを入れ、GPU/CPU リソースを節約する。
61     """
62     pipe = get_finbert()
63     results = []
64     batch_size = 32
65     for i in range(0, len(texts), batch_size):
66         batch = texts[i:i + batch_size]
67         try:
68             outputs = pipe(batch)
69             results.extend(outputs)
70         except Exception as e:
71             results.extend([{"label": "neutral", "score": 0.5}] * len(batch))
```

```

72         time.sleep(0.1)
73     return results
74
75
76 def fetch_news_newsapi(symbol: str, company_name: str = "") -> list[str]:
77     """NewsAPI から銘柄関連のニュース記事を取得する。
78
79     Args:
80         symbol: 銘柄シンボル。
81         company_name: 企業名（空文字の場合はシンボルのみで検索）。
82
83     Returns:
84         list[str]: 記事のタイトル+説明文のリスト。取得失敗時は空リスト。
85
86     Raises:
87         なし（例外は関数内で捕捉される）。
88     """
89     if not NEWSAPI_KEY:
90         return []
91
92     query = f"{symbol} stock"
93     if company_name:
94         query = f"({symbol} OR {company_name}) stock"
95
96     try:
97         resp = requests.get(
98             "https://newsapi.org/v2/everything",
99             params={
100                 "q": query,
101                 "language": "en",
102                 "sortBy": "publishedAt",
103                 "pageSize": 10,
104                 "apiKey": NEWSAPI_KEY,
105             },
106             timeout=10,
107         )
108         if resp.status_code == 200:
109             articles = resp.json().get("articles", [])
110             return [a.get("title", "") + ". " + (a.get("description") or "") for a in articles]
111         return []
112     except Exception:
113         return []
114
115
116 def fetch_news_finnhub(symbol: str) -> list[str]:
117     """Finnhub API から過去 7 日間の企業ニュースを取得する。
118
119     Args:
120         symbol: 銘柄シンボル。
121
122     Returns:
123         list[str]: 記事のヘッドライン+サマリーのリスト。取得失敗時は空リスト。
124
125     Raises:
126         なし（例外は関数内で捕捉される）。
127     """
128     if not FINNHUB_API_KEY:
129         return []
130
131     try:
132         resp = requests.get(
133             "https://finnhub.io/api/v1/company-news",
134             params={
135                 "symbol": symbol,
136                 "from": (datetime.now() - timedelta(days=7)).strftime("%Y-%m-%d"),
137                 "to": datetime.now().strftime("%Y-%m-%d"),
138                 "token": FINNHUB_API_KEY,
139             },
140             timeout=10,
141         )
142         if resp.status_code == 200:
143             articles = resp.json()
144             return [a.get("headline", "") + ". " + (a.get("summary") or "") for a in articles[:10]]
145         return []
146     except Exception:
147         return []
148

```

```

149
150 def get_next_batch_tickers(n: int) -> list[dict]:
151     """センチメント分析が未実行または次回取得時刻を過ぎた銘柄を取得する。
152
153     Args:
154         n: 取得する銘柄数。
155
156     Returns:
157         list[dict]: 各要素が {"id": int, "symbol": str, "name": str} のリスト。
158                     取得未実行の銘柄が優先される。
159
160     前提条件:
161         - sentiment_download_log テーブルが存在すること。
162     """
163     conn = get_connection()
164     cursor = conn.cursor()
165     cursor.execute("""
166         SELECT t.id, t.symbol, t.name
167         FROM tickers t
168         LEFT JOIN sentiment_download_log l ON t.id = l.ticker_id
169         WHERE t.is_active = 1
170               AND (l.last_downloaded_at IS NULL
171                    OR datetime(l.next_download_at) <= datetime('now'))
172         ORDER BY l.last_downloaded_at ASC NULLS FIRST
173         LIMIT ?
174     """, (n,))
175     rows = [dict(r) for r in cursor.fetchall()]
176     conn.close()
177     return rows
178
179
180 def save_sentiment_score(ticker_id: int, articles: list[str], scores: list[dict]):
181     """感情分析結果を indicators テーブルと sentiment_download_log に保存する。
182
183     Args:
184         ticker_id: 銘柄 ID。
185         articles: 分析対象の記事リスト。
186         scores: analyze_sentiment の戻り値 (label / score) 。
187
188     Returns:
189         None
190
191     Raises:
192         sqlite3.Error: DB 書き込みに失敗した場合。
193     """
194     if not articles or not scores:
195         return
196
197     pos_count = sum(1 for s in scores if s["label"].lower() == "positive")
198     neg_count = sum(1 for s in scores if s["label"].lower() == "negative")
199     neu_count = sum(1 for s in scores if s["label"].lower() == "neutral")
200     total = len(scores)
201
202     if total == 0:
203         return
204
205     score = ((pos_count - neg_count) / total) * 50 + 50
206     score = max(0, min(100, score))
207
208     details = {
209         "positive": pos_count,
210         "negative": neg_count,
211         "neutral": neu_count,
212         "total": total,
213         "scores": [{"text": a[:200], "label": s["label"], "confidence": round(s["score"], 4)}
214                    for a, s in zip(articles, scores)],
215     }
216
217     today = datetime.now().strftime("%Y-%m-%d")
218
219     conn = get_connection()
220     cursor = conn.cursor()
221
222     cursor.execute("""
223         INSERT OR REPLACE INTO indicators (ticker_id, date, rule_name, score, details_json)
224         VALUES (?, ?, 'sentiment', ?, ?)
225     """, (ticker_id, today, score, json.dumps(details, ensure_ascii=False)))

```



```

226
227 next_dl = (datetime.now() + timedelta(hours=SENTIMENT_MIN_INTERVAL_HOURS)).strftime("%Y-%m-%d %H:%M:%S")
228 cursor.execute("""
229     INSERT OR REPLACE INTO sentiment_download_log
230     (ticker_id, last_downloaded_at, next_download_at, articles_count)
231     VALUES (?, datetime('now'), ?, ?)
232     """, (ticker_id, next_dl, len(articles)))
233
234 conn.commit()
235 conn.close()
236
237
238 def download_next_batch(n: int = None):
239     """未処理銘柄のセンチメントデータをバッチダウンロード・分析・保存する。
240
241     NewsAPI（優先）→ Finnhub（フォールバック）の順でニュースを取得し、
242     FinBERT で感情分析を行った結果を DB に保存する。
243
244     Args:
245         n: バッチサイズ（デフォルトは SENTIMENT_BATCH_SIZE）。
246
247     Returns:
248         None
249
250     注意:
251     - ニュースが取得できなかった銘柄は neutral（スコア 50.0）として記録される。
252     """
253     if n is None:
254         n = SENTIMENT_BATCH_SIZE
255
256     batch = get_next_batch_tickers(n)
257     if not batch:
258         print("No tickers pending sentiment download.")
259         return
260
261     for item in batch:
262         ticker_id = item["id"]
263         symbol = item["symbol"]
264         name = item.get("name") or ""
265
266         print(f"Sentiment: {symbol} ({name})")
267
268         articles = fetch_news_newsapi(symbol, name)
269         if not articles:
270             articles = fetch_news_finnhub(symbol)
271
272         if not articles:
273             save_sentiment_score(ticker_id, [], [{"label": "neutral", "score": 0.5}])
274             print(f" -> No news found, default neutral")
275             continue
276
277         scores = analyze_sentiment(articles)
278         save_sentiment_score(ticker_id, articles, scores)
279
280         pos = sum(1 for s in scores if s["label"].lower() == "positive")
281         neg = sum(1 for s in scores if s["label"].lower() == "negative")
282         print(f" -> {len(articles)} articles, pos={pos}, neg={neg}")
283
284         time.sleep(1)
285
286     print(f"Sentiment batch complete: {len(batch)} tickers")

```

schema.py

stockChecker¥db¥schema.py

175 lines | 6,336 bytes

```
1  """データベーススキーマ定義と接続管理
2
3  データベースの初期化（テーブル作成・インデックス作成）および
4  接続取得・エラーログ記録の共通関数を提供する。
5
6  テーブル一覧:
7      - tickers: 銘柄マスタ
8      - prices: 日次株価データ
9      - financials: 財務データ
10     - indicators: ルール別指標スコア
11     - backtest_results: バックテスト実行結果
12     - sentiment_download_log: センチメント取得ログ
13     - error_log: エラーログ
14 """
15
16 import sqlite3
17 from typing import Optional
18 import config
19
20
21 # sqlite3.Row を行ファクトリに設定すると、カラム名で値にアクセスできる辞書ライクな行オブジェクトが返る
22 # PRAGMA 設定: WAL モード（読み取り中でも書込可能）+ 外部キー制約 ON（デフォルト OFF）
23 def get_connection() -> sqlite3.Connection:
24     """SQLite データベース接続を取得する。
25
26     接続には Row ファクトリ・WAL モード・外部キー制約が設定される。
27
28     Returns:
29         sqlite3.Connection: 設定済みのデータベース接続オブジェクト。
30
31     Raises:
32         sqlite3.Error: DB ファイルのオープンに失敗した場合。
33     """
34     conn = sqlite3.connect(config.DB_PATH)
35     conn.row_factory = sqlite3.Row
36     conn.execute("PRAGMA journal_mode=WAL")
37     conn.execute("PRAGMA foreign_keys=ON")
38     return conn
39
40
41 # executescript() は複数の SQL 文を；区切りで一括実行できる（execute は 1 文のみ）
42 def init_db() -> None:
43     """データベースを初期化し、全テーブル・インデックスを作成する。
44
45     既存のテーブルが存在する場合は CREATE IF NOT EXISTS によりスキップされ、
46     データは保持される。
47
48     Args:
49         なし
50
51     Returns:
52         None
53
54     Raises:
55         sqlite3.Error: テーブル作成に失敗した場合。
56     """
57     conn = get_connection()
58     cursor = conn.cursor()
59
60     cursor.executescript("""
61         CREATE TABLE IF NOT EXISTS tickers (
62             id            INTEGER PRIMARY KEY AUTOINCREMENT,
63             symbol        TEXT NOT NULL UNIQUE,
64             name          TEXT,
65             market        TEXT NOT NULL,
66             sector        TEXT,
67             market_cap    REAL,
68             is_active     INTEGER DEFAULT 1,
69             created_at    TEXT DEFAULT (datetime('now')),
70             updated_at    TEXT DEFAULT (datetime('now'))
71         );
```

```

72 CREATE INDEX IF NOT EXISTS idx_tickers_market ON tickers(market);
73 CREATE INDEX IF NOT EXISTS idx_tickers_symbol ON tickers(symbol);
74
75 CREATE TABLE IF NOT EXISTS prices (
76     ticker_id    INTEGER NOT NULL,
77     date         TEXT NOT NULL,
78     open         REAL,
79     high         REAL,
80     low          REAL,
81     close        REAL,
82     volume       INTEGER,
83     dividends    REAL DEFAULT 0,
84     splits       REAL DEFAULT 1,
85     PRIMARY KEY (ticker_id, date),
86     FOREIGN KEY (ticker_id) REFERENCES tickers(id)
87 );
88 CREATE INDEX IF NOT EXISTS idx_prices_date ON prices(date);
89
90 CREATE TABLE IF NOT EXISTS financials (
91     ticker_id    INTEGER NOT NULL,
92     fiscal_year  INTEGER NOT NULL,
93     revenue      REAL,
94     operating_income REAL,
95     net_income   REAL,
96     total_assets REAL,
97     total_equity REAL,
98     cash_flow    REAL,
99     per          REAL,
100    pbr           REAL,
101    roe           REAL,
102    dividend_yield REAL,
103    source        TEXT DEFAULT 'yfinance',
104    PRIMARY KEY (ticker_id, fiscal_year),
105    FOREIGN KEY (ticker_id) REFERENCES tickers(id)
106 );
107
108 CREATE TABLE IF NOT EXISTS indicators (
109     ticker_id    INTEGER NOT NULL,
110     date         TEXT NOT NULL,
111     rule_name    TEXT NOT NULL,
112     score        REAL,
113     details_json TEXT,
114     PRIMARY KEY (ticker_id, date, rule_name),
115     FOREIGN KEY (ticker_id) REFERENCES tickers(id)
116 );
117 CREATE INDEX IF NOT EXISTS idx_indicators_rule_date ON indicators(rule_name, date);
118
119 CREATE TABLE IF NOT EXISTS backtest_results (
120     id           INTEGER PRIMARY KEY AUTOINCREMENT,
121     base_date    TEXT NOT NULL,
122     rule_name    TEXT NOT NULL,
123     ticker_id    INTEGER NOT NULL,
124     score        REAL,
125     rank         INTEGER,
126     executed_at  TEXT DEFAULT (datetime('now')),
127     FOREIGN KEY (ticker_id) REFERENCES tickers(id)
128 );
129 CREATE INDEX IF NOT EXISTS idx_backtest_base ON backtest_results(base_date, rule_name);
130
131 CREATE TABLE IF NOT EXISTS sentiment_download_log (
132     ticker_id    INTEGER PRIMARY KEY,
133     last_downloaded_at TEXT,
134     next_download_at TEXT,
135     articles_count INTEGER DEFAULT 0,
136     error_count   INTEGER DEFAULT 0,
137     FOREIGN KEY (ticker_id) REFERENCES tickers(id)
138 );
139
140 CREATE TABLE IF NOT EXISTS error_log (
141     id           INTEGER PRIMARY KEY AUTOINCREMENT,
142     timestamp    TEXT DEFAULT (datetime('now')),
143     operation     TEXT,
144     ticker_id    INTEGER,
145     error_message TEXT,
146     FOREIGN KEY (ticker_id) REFERENCES tickers(id)
147 );
148 """

```

```

149
150     conn.commit()
151     conn.close()
152
153
154 def log_error(operation: str, ticker_id: Optional[int], message: str) -> None:
155     """エラーログを error_log テーブルに記録する。
156
157     Args:
158         operation: エラーが発生した操作名 (例: "download_price")。
159         ticker_id: 関連する銘柄 ID (関連がない場合は None)。
160         message: エラーメッセージ。
161
162     Returns:
163         None
164
165     Raises:
166         sqlite3.Error: ログの書き込みに失敗した場合。
167     """
168     conn = get_connection()
169     cursor = conn.cursor()
170     cursor.execute(
171         "INSERT INTO error_log (operation, ticker_id, error_message) VALUES (?, ?, ?)",
172         (operation, ticker_id, message),
173     )
174     conn.commit()
175     conn.close()

```

tickers.py

stockChecker¥db¥tickers.py

197 lines | 7,882 bytes

```
1 """ 銘柄一覧の取得・管理
2
3 JPX（日本取引所）・Wikipedia（S&P500）から銘柄リストを取得し、
4 tickers テーブルを更新する。厳選 US 銘柄の定数リストも保持する。
5 """
6
7 import io
8 import csv
9 import requests
10 import pandas as pd
11 from typing import List, Tuple, Optional
12
13 from config import JPX_TICKER_URLS
14 from db.schema import get_connection
15
16 CURATED_US_TICKERS: List[Tuple[str, str]] = [
17     ("AAPL", "Apple Inc."), ("MSFT", "Microsoft Corp."), ("GOOGL", "Alphabet Inc."),
18     ("AMZN", "Amazon.com Inc."), ("NVDA", "NVIDIA Corp."), ("META", "Meta Platforms Inc."),
19     ("TSLA", "Tesla Inc."), ("BRK-B", "Berkshire Hathaway"), ("JPM", "JPMorgan Chase"),
20     ("V", "Visa Inc."), ("JNJ", "Johnson & Johnson"), ("WMT", "Walmart Inc."),
21     ("PG", "Procter & Gamble"), ("MA", "Mastercard Inc."), ("UNH", "UnitedHealth Group"),
22     ("HD", "Home Depot Inc."), ("DIS", "Walt Disney Co."), ("BAC", "Bank of America"),
23     ("ADBE", "Adobe Inc."), ("NFLX", "Netflix Inc."), ("CRM", "Salesforce Inc."),
24     ("KO", "Coca-Cola Co."), ("PEP", "PepsiCo Inc."), ("XOM", "Exxon Mobil Corp."),
25     ("CVX", "Chevron Corp."), ("ABBV", "AbbVie Inc."), ("COST", "Costco Wholesale"),
26     ("PYPL", "PayPal Holdings"), ("AMD", "Advanced Micro Devices"),
27     ("INTC", "Intel Corp."), ("IBM", "IBM Corp."), ("ORCL", "Oracle Corp."),
28     ("CSCO", "Cisco Systems"), ("QCOM", "Qualcomm Inc."), ("TXN", "Texas Instruments"),
29     ("NFLX", "Netflix Inc."), ("CMCSA", "Comcast Corp."), ("T", "AT&T Inc."),
30     ("VZ", "Verizon Communications"), ("MRK", "Merck & Co."), ("PFE", "Pfizer Inc."),
31     ("TMO", "Thermo Fisher Scientific"), ("AVGO", "Broadcom Inc."),
32     ("ACN", "Accenture PLC"), ("DHR", "Danaher Corp."), ("LIN", "Linde PLC"),
33     ("NEE", "NextEra Energy"), ("BA", "Boeing Co."), ("GE", "General Electric"),
34     ("CAT", "Caterpillar Inc."),
35 ]
36
37
38 def download_jpx_list() -> List[Tuple[str, str, str]]:
39     """ JPX（日本取引所）から上場銘柄一覧をダウンロードする。
40
41     プライム・スタンダード・グロースの各市場別 CSV を取得し、
42     （シンボル、銘柄名、"japan"）のタプルリストとして返す。
43
44     Returns:
45         List[Tuple[str, str, str]]: （symbol, name, market）のタプルリスト。
46         ダウンロード失敗時は空リスト。
47
48     Raises:
49         requests.RequestException: HTTP 通信エラー（関数内で捕捉・出力済み）。
50     """
51     # io.StringIO(resp.text) はテキストをファイルオブジェクトのように扱う（csv.reader がファイル入力を想定しているため）
52     tickers = []
53     for market, url in JPX_TICKER_URLS.items():
54         try:
55             resp = requests.get(url, timeout=30)
56             resp.encoding = "shift_jis"
57             reader = csv.reader(io.StringIO(resp.text))
58             header_skipped = False
59             for row in reader:
60                 if not header_skipped:
61                     header_skipped = True
62                     continue
63                 if len(row) < 2:
64                     continue
65                 code = row[0].strip()
66                 name = row[1].strip()
67                 symbol = f"{code}.T"
68                 tickers.append((symbol, name, "japan"))
69         except Exception as e:
70             print(f"Failed to download JPX {market}: {e}")
71     return tickers
```

```

72
73
74 def fetch_sp500_from_wikipedia() -> List[Tuple[str, str]]:
75     """Wikipedia の S&P 500 構成銘柄一覧をスクレイピングする。
76
77     Returns:
78         List[Tuple[str, str]]: (symbol, name) のタプルリスト。
79         スクレイピング失敗時は空リスト。
80
81     注意:
82         - Wikipedia のテーブル構造が変更された場合、解析に失敗する可能性がある。
83     """
84     # pd.read_html(url) は HTML 内の <table> を自動検出し DataFrame のリストとして返す
85     try:
86         url = "https://en.wikipedia.org/wiki/List_of_S%26P_500_companies"
87         tables = pd.read_html(url)
88         df = tables[0]
89         tickers = []
90         for _, row in df.iterrows():
91             symbol = row["Symbol"].strip()
92             name = row["Security"].strip()
93             if "." in symbol:
94                 symbol = symbol.replace(".", "-")
95             tickers.append((symbol, name))
96         return tickers
97     except Exception as e:
98         print(f"Failed to fetch S&P 500 from Wikipedia: {e}")
99         return []
100
101
102 # ON CONFLICT(symbol) DO UPDATE SET - SQLite の UPSERT 構文
103 # INSERT で UNIQUE 制約違反が発生した場合は UPDATE にフォールバックする
104 def update_ticker_list() -> None:
105     """tickers テーブルを最新の銘柄一覧で更新する。
106
107     S&P 500・厳選 US 銘柄・JPX 上場銘柄の全リストを取得し、
108     INSERT OR ON CONFLICT でマージする（既存は updated_at 更新、
109     新規は INSERT、非掲載銘柄は is_active が更新されない）。
110
111     Returns:
112         None
113
114     Raises:
115         sqlite3.Error: DB 操作に失敗した場合。
116     """
117     conn = get_connection()
118     cursor = conn.cursor()
119
120     total = 0
121
122     sp500 = fetch_sp500_from_wikipedia()
123     for symbol, name in sp500:
124         cursor.execute("""
125             INSERT INTO tickers (symbol, name, market, updated_at)
126             VALUES (?, ?, 'us', datetime('now'))
127             ON CONFLICT(symbol) DO UPDATE SET
128                 name = excluded.name,
129                 market = excluded.market,
130                 is_active = 1,
131                 updated_at = datetime('now')
132             """, (symbol, name))
133     total += len(sp500)
134     print(f"US (S&P500) tickers: {len(sp500)}")
135
136     for symbol, name in CURATED_US_TICKERS:
137         cursor.execute("""
138             INSERT INTO tickers (symbol, name, market, updated_at)
139             VALUES (?, ?, 'us', datetime('now'))
140             ON CONFLICT(symbol) DO UPDATE SET
141                 name = excluded.name,
142                 market = excluded.market,
143                 is_active = 1,
144                 updated_at = datetime('now')
145             """, (symbol, name))
146     total += len(CURATED_US_TICKERS)
147
148     jp_tickers = download_jpx_list()

```

```

149     for symbol, name, market in jp_tickers:
150         cursor.execute("""
151             INSERT INTO tickers (symbol, name, market, updated_at)
152             VALUES (?, ?, ?, datetime('now'))
153             ON CONFLICT(symbol) DO UPDATE SET
154                 name = excluded.name,
155                 market = excluded.market,
156                 is_active = 1,
157                 updated_at = datetime('now')
158             """, (symbol, name, market))
159     total += len(jp_tickers)
160     print(f"JP tickers: {len(jp_tickers)}")
161
162     conn.commit()
163     conn.close()
164     print(f"Total tickers updated: {total}")
165
166 def get_all_active_tickers() -> List[dict]:
167     """アクティブな全銘柄の一覧を取得する。
168
169     Returns:
170         List[dict]: 各要素が {"id": int, "symbol": str, "name": str, "market": str} のリスト。
171                     シンボル順でソートされる。
172     """
173
174     conn = get_connection()
175     cursor = conn.cursor()
176     cursor.execute("SELECT id, symbol, name, market FROM tickers WHERE is_active = 1 ORDER BY symbol")
177     rows = [dict(r) for r in cursor.fetchall()]
178     conn.close()
179     return rows
180
181
182 def get_ticker_by_symbol(symbol: str) -> Optional[dict]:
183     """指定されたシンボルに対応する銘柄情報を取得する。
184
185     Args:
186         symbol: 銘柄シンボル（例: "AAPL"）。
187
188     Returns:
189         Optional[dict]: {"id": int, "symbol": str, "name": str, "market": str}。
190                         該当銘柄が存在しない場合は None。
191     """
192
193     conn = get_connection()
194     cursor = conn.cursor()
195     cursor.execute("SELECT id, symbol, name, market FROM tickers WHERE symbol = ?", (symbol,))
196     row = cursor.fetchone()
197     conn.close()
198     return dict(row) if row else None

```

`__init__.py`

stockChecker¥rules¥__init__.py

1 lines | 0 bytes

1

anomaly.py

stockChecker¥rules¥anomaly.py

86 lines | 3,069 bytes

```
1 """異常検知ルール（出来高・ボラティリティ急増銘柄を検出）
2
3 直近期間の出来高とボラティリティが過去と比較して急増しているかを測定し、
4 大きな値動きの予兆としてスコアリングする。
5 """
6
7 import numpy as np
8 from rules.base import BaseRule
9
10
11 class AnomalyRule(BaseRule):
12     """異常検知ルール
13
14     直近期間（20 日）の出来高・ボラティリティが過去と比較して急増しているかを測定する。
15     """
16
17     name = "異常検知"
18     description = "出来高やボラティリティが急増した銘柄は、短期的に大きく動きやすい"
19
20     def need_financials(self) -> bool:
21         """価格データ（出来高・終値）のみで計算可能なため、財務データは不要。
22
23         Returns:
24             bool: 常に False。
25         """
26         return False
27
28     def calculate(self, ticker_id: int, base_date: str) -> float:
29         """出来高比率とボラティリティ比率から異常検知スコアを計算する。
30
31         直近 20 日の平均出来高 / 過去 126 日の平均出来高、および
32         直近 20 日のボラティリティ / 過去のボラティリティを測定する。
33
34         Args:
35             ticker_id: 評価対象の銘柄 ID。
36             base_date: 基準日（"YYYY-MM-DD" 形式）。
37
38         Returns:
39             float: 0~100 のスコア。急増度が高いほど高スコア。
40                 最低 40 営業日のデータがない場合は 40.0 を返す。
41
42         注意:
43             - 出来高 3 倍以上 +60、2 倍以上 +40、1.5 倍以上 +20。
44             - ボラティリティ 2 倍以上 +40、1.5 倍以上 +20。
45         """
46         df = self.get_prices(ticker_id, base_date, lookback_days=126)
47         if df is None or len(df) < 40:
48             return 40.0
49
50         volumes = df["volume"].values.astype(float)
51         closes = df["close"].values
52
53         if np.std(volumes) == 0:
54             return 40.0
55
56         # 直近 20 日の出来高平均 ÷ 過去の出来高平均 → 出来高急増率
57         # [+]1e-10 はゼロ除算防止（すべての値が 0 でも割り算が成立するよう微小値を足す）
58         recent_vol = volumes[-20:]
59         past_vol = volumes[:-20]
60
61         if len(past_vol) < 10:
62             return 40.0
63
64         vol_ratio = np.mean(recent_vol) / (np.mean(past_vol) + 1e-10)
65
66         # 対数収益率の標準偏差でボラティリティを測定
67         log_returns = np.log(closes[1:] / closes[:-1])
68         recent_vola = np.std(log_returns[-20:])
69         past_vola = np.std(log_returns[:-20])
70
71         vola_ratio = recent_vola / (past_vola + 1e-10) if len(log_returns) > 20 else 1.0
```

```
72
73     score = 0
74     if vol_ratio > 3:
75         score += 60
76     elif vol_ratio > 2:
77         score += 40
78     elif vol_ratio > 1.5:
79         score += 20
80
81     if vola_ratio > 2:
82         score += 40
83     elif vola_ratio > 1.5:
84         score += 20
85
86     return min(100, score + 10)
```

base.py

stockChecker¥rules¥base.py

185 lines | 7,176 bytes

```
1 """バックテストルールの抽象基底クラスと共通ユーティリティ
2
3 全ルールが継承すべき BaseRule 抽象クラスと、スコア正規化関数を提供する。
4 BaseRule は価格・財務・指標データの DB 取得メソッドを共通実装として持つ。
5 """
6
7 from abc import ABC, abstractmethod
8 from datetime import datetime, timedelta
9 from typing import Optional
10 import pandas as pd
11
12 from db.schema import get_connection
13
14
15 # ABC = Abstract Base Class (抽象基底クラス) - 直接インスタンス化できず、継承して使うことを強制する
16 # @abstractmethod のメソッドはサブクラスで必ずオーバーライドしなければならない (忘れるとエラー)
17 class BaseRule(ABC):
18     name: str = ""
19     description: str = ""
20
21     @abstractmethod
22     def need_financials(self) -> bool:
23         """当該ルールが財務データを必要とするかどうかを返す。
24
25         Returns:
26             bool: 財務データが必要な場合は True、不要な場合は False。
27
28         注意:
29             - 価格データのみで計算可能なルールは False を返す。
30             - 財務指標 (PER・PBR・ROE 等) を用いるルールは True を返す。
31         """
32         pass
33
34     @abstractmethod
35     def calculate(self, ticker_id: int, base_date: str) -> float:
36         """指定された銘柄・基準日におけるスコアを計算する。
37
38         Args:
39             ticker_id: 評価対象の銘柄 ID (tickers テーブルの主キー)。
40             base_date: 基準日 ("YYYY-MM-DD" 形式)。
41
42         Returns:
43             float: 0.0~100.0 の範囲に正規化されたスコア。
44                 値が大きいほど有望な銘柄と判定される。
45
46         Raises:
47             ValueError: 必要なデータが不足している場合。
48         """
49         pass
50
51 # @abstractmethod がない = 共通の実装済みメソッド。サブクラスはそのまま使える (必要ならオーバーライドも可)
52 # pandas の read_sql_query は SQL の結果を直接 DataFrame として返す (ループ不要で便利)
53 def get_prices(self, ticker_id: int, base_date: str, lookback_days: int = 365) -> Optional[pd.DataFrame]:
54     """指定銘柄の価格データをデータベースから取得する。
55
56     基準日から過去 lookback_days 日間の日次価格 (日付・始値・高値・安値・終値・出来高) を
57     昇順で返す。
58
59     Args:
60         ticker_id: 銘柄 ID。
61         base_date: 基準日 ("YYYY-MM-DD" 形式)。
62         lookback_days: 取得する過去日数 (デフォルト 365 日)。
63
64     Returns:
65         Optional[pd.DataFrame]: カラム ['date', 'open', 'high', 'low', 'close', 'volume'] を持つ
66                                 DataFrame。データが存在しない場合は None。
67
68     前提条件:
69         - prices テーブルに該当 ticker_id のデータが存在すること。
70     """
71     conn = get_connection()
```

```

72     end = datetime.strptime(base_date, "%Y-%m-%d")
73     start = end - timedelta(days=lookback_days)
74
75     query = """
76         SELECT date, open, high, low, close, volume
77         FROM prices
78         WHERE ticker_id = ? AND date >= ? AND date <= ?
79         ORDER BY date ASC
80     """
81     df = pd.read_sql_query(query, conn, params=(ticker_id, start.strftime("%Y-%m-%d"), base_date))
82     conn.close()
83
84     if df.empty:
85         return None
86     return df
87
88 def get_financial(self, ticker_id: int, fiscal_year: int) -> Optional[dict]:
89     """指定銘柄・指定年度の財務データを取得する。
90
91     Args:
92         ticker_id: 銘柄 ID。
93         fiscal_year: 取得対象の会計年度（例：2023）。
94
95     Returns:
96         Optional[dict]: financials テーブルの全カラムを含む辞書。
97             データが存在しない場合は None。
98
99     前提条件:
100         - financials テーブルに該当データが存在すること。
101     """
102     conn = get_connection()
103     cursor = conn.cursor()
104     cursor.execute(
105         "SELECT * FROM financials WHERE ticker_id = ? AND fiscal_year = ?",
106         (ticker_id, fiscal_year),
107     )
108     row = cursor.fetchone()
109     conn.close()
110     return dict(row) if row else None
111
112 def get_latest_financial(self, ticker_id: int, base_date: str) -> Optional[dict]:
113     """指定銘柄の最新財務データを基準日以前から取得する。
114
115     Args:
116         ticker_id: 銘柄 ID。
117         base_date: 基準日（"YYYY-MM-DD" 形式）。
118
119     Returns:
120         Optional[dict]: 最新の financials レコードの辞書。
121             データが存在しない場合は None。
122
123     注意:
124         - 基準日の年以前の最新年度データを返す（決算発表ラグを考慮）。
125     """
126     base_year = int(base_date[:4])
127     conn = get_connection()
128     cursor = conn.cursor()
129     cursor.execute("""
130         SELECT * FROM financials
131         WHERE ticker_id = ? AND fiscal_year <= ?
132         ORDER BY fiscal_year DESC
133         LIMIT 1
134     """, (ticker_id, base_year))
135     row = cursor.fetchone()
136     conn.close()
137     return dict(row) if row else None
138
139 def get_indicator(self, ticker_id: int, date: str, rule_name: str) -> Optional[float]:
140     """事前計算済みのルール別指標値をデータベースから取得する。
141
142     Args:
143         ticker_id: 銘柄 ID。
144         date: 取得基準日（"YYYY-MM-DD" 形式）。
145         rule_name: ルール名称。
146
147     Returns:
148         Optional[float]: 指標スコア。データが存在しない場合は None。

```

```

149
150     注意:
151     - indicators テーブルから指定日以前の最新スコアを取得する。
152     """
153     conn = get_connection()
154     cursor = conn.cursor()
155     cursor.execute(
156         "SELECT score FROM indicators WHERE ticker_id = ? AND date <= ? AND rule_name = ? ORDER BY date DESC LIMIT 1",
157         (ticker_id, date, rule_name),
158     )
159     row = cursor.fetchone()
160     conn.close()
161     return row["score"] if row else None
162
163
164 def normalize_score(raw_score: float, min_val: float, max_val: float) -> float:
165     """生のスコア値を 0~100 の範囲に線形正規化する。
166
167     最小値と最大値が等しい場合は中央値 50.0 を返す。
168     正規化後は [0, 100] の範囲にクリップされる。
169
170     Args:
171         raw_score: 正規化前の生スコア。
172         min_val: 全銘柄中の最小スコア。
173         max_val: 全銘柄中の最大スコア。
174
175     Returns:
176         float: 0.0~100.0 に正規化されたスコア。
177
178     使用例:
179     >>> normalize_score(150.0, 100.0, 200.0)
180     50.0
181     """
182     if max_val == min_val:
183         return 50.0
184     normalized = (raw_score - min_val) / (max_val - min_val) * 100
185     return max(0, min(100, normalized))

```

low_vol.py

stockChecker¥rules¥low_vol.py

68 lines | 2,557 bytes

```
1 """低ボラティリティ効果（値動きが小さい株のリターンが高い傾向）を測定するルール
2
3 年率換算したヒストリカル・ボラティリティを基準にスコアリングする。
4 """
5
6 import numpy as np
7 from rules.base import BaseRule
8
9
10 class LowVolRule(BaseRule):
11     """低ボラティリティ効果ルール
12
13     過去 1 年の年率換算ヒストリカル・ボラティリティが低いほど高スコアを割り当てる。
14     """
15
16     name = "低ボラティリティ"
17     description = "値動きが小さい株のほうが、長期的にリターンが高い"
18
19     def need_financials(self) -> bool:
20         """価格データのみで計算可能なため、財務データは不要。
21
22         Returns:
23             bool: 常に False。
24         """
25         return False
26
27     def calculate(self, ticker_id: int, base_date: str) -> float:
28         """年率換算ヒストリカル・ボラティリティから低ボラティリティスコアを計算する。
29
30         Args:
31             ticker_id: 評価対象の銘柄 ID。
32             base_date: 基準日 ("YYYY-MM-DD" 形式)。
33
34         Returns:
35             float: 0~100 のスコア。ボラティリティが低いほど高スコア。
36                 最低 60 営業日のデータがない場合は 40.0 を返す。
37
38         注意:
39             - 対数収益率の標準偏差 × sqrt(252) で年率換算。
40         """
41         df = self.get_prices(ticker_id, base_date, lookback_days=252)
42         if df is None or len(df) < 60:
43             return 40.0
44
45         # closes[1:] = 2 番目以降（今日）、closes[:-1] = 最後以外（昨日）
46         # 除算で「今日 ÷ 昨日」のリターン比を計算し、np.log で対数収益率に変換
47         # 対数収益率は「連続複利」の仮定に基づくファイナンスの標準手法
48         closes = df["close"].values
49         log_returns = np.log(closes[1:] / closes[:-1])
50         hv = np.std(log_returns) * np.sqrt(252)
51
52         annualized_vol = hv
53
54         if annualized_vol <= 0:
55             return 40.0
56
57         # 年率 15% 未満 → 「低ボラ」= 90 点。50% 超 → 「高ボラ」= 15 点
58         # 閾値は過去の市場平均（年率 15~20%）を参考に設定
59         if annualized_vol < 0.15:
60             return 90.0
61         elif annualized_vol < 0.25:
62             return 75.0
63         elif annualized_vol < 0.35:
64             return 55.0
65         elif annualized_vol < 0.50:
66             return 35.0
67         else:
68             return 15.0
```

momentum.py

stockChecker¥rules¥momentum.py

61 lines | 2,390 bytes

```
1  """モメンタム効果（過去の上昇が継続する傾向）を測定するルール
2
3  過去3ヶ月・6ヶ月・12ヶ月のリターンを加重平均し、0-100のスコアに正規化する。
4  """
5
6  import numpy as np
7  from rules.base import BaseRule, normalize_score
8
9
10 class MomentumRule(BaseRule):
11     """モメンタム効果ルール
12
13     過去 3・6・12 ヶ月のリターンを加重平均し、上昇トレンド銘柄を高スコア評価する。
14     """
15
16     name = "モメンタム"
17     description = "過去3～12ヶ月で上昇した銘柄は、その後も上昇しやすい"
18
19     def need_financials(self) -> bool:
20         """価格データのみで計算可能なため、財務データは不要。
21
22         Returns:
23             bool: 常に False。
24         """
25         return False
26
27     def calculate(self, ticker_id: int, base_date: str) -> float:
28         """過去 3・6・12 ヶ月のリターンを加重平均してスコアを計算する。
29
30         Args:
31             ticker_id: 評価対象の銘柄 ID。
32             base_date: 基準日 ("YYYY-MM-DD" 形式)。
33
34         Returns:
35             float: 0～100 のスコア。値が大きいほど上昇モメンタムが強い。
36                 最低 60 営業日のデータがない場合は 40.0 を返す。
37
38         注意:
39             - 3 ヶ月・6 ヶ月・12 ヶ月のリターンを 3:3:4 の比率で加重する。
40         """
41         df = self.get_prices(ticker_id, base_date, lookback_days=365)
42         if df is None or len(df) < 60:
43             return 40.0
44
45         closes = df["close"].values
46         periods = [63, 126, 252]
47         # 各期間のリターン計算: closes[-1] は最新終値、closes[-p] は p 営業日前の終値
48         # 配列の末尾が「最新」、先頭が「最古」であるのに注意
49         for p in periods:
50             if len(closes) >= p:
51                 ret = (closes[-1] / closes[-p]) - 1
52                 scores.append(ret)
53             else:
54                 ret = (closes[-1] / closes[0]) - 1
55                 scores.append(ret)
56
57         momentum_3m, momentum_6m, momentum_12m = scores
58         # 短期ほど将来予測にノイズが多いため、12 ヶ月に最も大きい重み 0.4 を割り当てる
59         weighted = momentum_3m * 0.3 + momentum_6m * 0.3 + momentum_12m * 0.4
60
61         return normalize_score(weighted, -0.3, 0.5)
```

pead.py

stockChecker¥rules¥pead.py

95 lines | 3,195 bytes

```
1 """PEAD（決算サプライズ後のドリフト）を測定するルール
2
3 yfinanceから直近の決算サプライズ率を取得し、ポジティブ・サプライズほど高スコアをつける。
4 """
5
6 import yfinance as yf
7 import pandas as pd
8 from rules.base import BaseRule
9 from db.schema import get_connection
10
11
12 class PeadRule(BaseRule):
13     """PEAD（Post-Earnings Announcement Drift）ルール
14
15     直近の決算サプライズ率を yfinance から取得し、
16     ポジティブ・サプライズほど高スコアを割り当てる。
17     """
18
19     name = "PEAD"
20     description = "良い決算を出した企業は、発表後数週間～数ヶ月上昇しやすい"
21
22     def need_financials(self) -> bool:
23         """決算サプライズは yfinance の earnings_dates から直接取得するため不要。
24
25         Returns:
26             bool: 常に False。
27         """
28         return False
29
30     def calculate(self, ticker_id: int, base_date: str) -> float:
31         """直近決算のサプライズ率から PEAD スコアを計算する。
32
33         Args:
34             ticker_id: 評価対象の銘柄 ID。
35             base_date: 基準日（"YYYY-MM-DD" 形式）。
36
37         Returns:
38             float: 0～100 のスコア。サプライズ率が高いほど高スコア。
39                 データ取得失敗時は 40.0 を返す。
40
41         注意:
42             - yfinance API のレート制限に影響される可能性がある。
43         """
44         conn = get_connection()
45         cursor = conn.cursor()
46         cursor.execute("SELECT symbol FROM tickers WHERE id = ?", (ticker_id,))
47         row = cursor.fetchone()
48         conn.close()
49
50         if not row:
51             return 40.0
52
53         symbol = row["symbol"]
54
55         # ticker.earnings_dates は yfinance が提供する決算発表日とサプライズ率のテーブル
56         # reset_index() で日付インデックスを通常カラムに変換し、日付フィルタを可能にする
57         try:
58             ticker = yf.Ticker(symbol)
59             earnings = ticker.earnings_dates
60
61             if earnings is None or earnings.empty:
62                 return 40.0
63
64             earnings = earnings.reset_index()
65             earnings["Earnings Date"] = pd.to_datetime(earnings["Earnings Date"])
66             earnings = earnings.sort_values("Earnings Date", ascending=False)
67
68             # 基準日以前の直近決算のみを対象（未来の決算日は除外）
69             recent = earnings[earnings["Earnings Date"] <= base_date]
70             if recent.empty:
71                 return 40.0
```



```

72
73     latest = recent.iloc[0]
74
75     if "Surprise(%)" not in latest.index:
76         return 40.0
77
78     surprise = latest.get("Surprise(%)", 0)
79     if pd.isna(surprise):
80         return 40.0
81
82     eps_pct = float(surprise)
83
84     if eps_pct > 10:
85         return 85.0
86     elif eps_pct > 5:
87         return 70.0
88     elif eps_pct > 0:
89         return 55.0
90     elif eps_pct > -5:
91         return 40.0
92     else:
93         return 20.0
94 except Exception:
95     return 40.0

```

quality.py

stockChecker¥rules¥quality.py

76 lines | 2,755 bytes

```
1 """クオリティ効果（高品質企業が安定したリターンを生む傾向）を測定するルール
2
3 ROE・自己資本比率・営業利益率の3要素を加重平均してスコア化する。
4 """
5
6 from rules.base import BaseRule
7
8
9 class QualityRule(BaseRule):
10     """クオリティ効果ルール
11
12     ROE・自己資本比率・営業利益率から企業の質を総合評価する。
13     """
14
15     name = "クオリティ"
16     description = "ROE・自己資本比率が高い質の良い企業は長期的に安定したリターンを生む"
17
18     def need_financials(self) -> bool:
19         """ROE・自己資本・営業利益を参照するため、財務データは必須。
20
21         Returns:
22             bool: 常に True。
23         """
24         return True
25
26     def calculate(self, ticker_id: int, base_date: str) -> float:
27         """ROE・自己資本比率・営業利益率を加重平均してクオリティスコアを計算する。
28
29         Args:
30             ticker_id: 評価対象の銘柄 ID。
31             base_date: 基準日（"YYYY-MM-DD" 形式）。
32
33         Returns:
34             float: 0~100 のスコア。値が大きいくほど企業の質が高い。
35                    財務データがない場合は 40.0 を返す。
36
37         注意:
38             - ROE は 40%、自己資本比率 30%、営業利益率 30% で加重。
39         """
40         fin = self.get_latest_financial(ticker_id, base_date)
41         if fin is None:
42             return 40.0
43
44         roe = fin.get("roe")
45         total_equity = fin.get("total_equity")
46         total_assets = fin.get("total_assets")
47         op_income = fin.get("operating_income")
48         revenue = fin.get("revenue")
49
50         score = 0
51         weights = 0
52
53         # ROE（自己資本利益率）：高いほど効率的に利益を生んでいる。最重視 40%
54         if roe is not None and roe > 0:
55             roe_score = min(100, roe * 200)
56             score += roe_score * 0.4
57             weights += 0.4
58
59         # 自己資本比率 = 自己資本 ÷ 総資産。高すぎるより適度が理想だが単純化
60         if total_equity is not None and total_assets is not None and total_assets > 0:
61             eq_ratio = total_equity / total_assets
62             eq_score = min(100, eq_ratio * 150)
63             score += eq_score * 0.3
64             weights += 0.3
65
66         # 営業利益率 = 営業利益 ÷ 売上高。本業の収益性を示す
67         if op_income is not None and revenue is not None and revenue > 0:
68             margin = op_income / revenue
69             margin_score = min(100, margin * 400)
70             score += margin_score * 0.3
71             weights += 0.3
```

```
72
73     if weights == 0:
74         return 40.0
75
76     return score / weights
```

runner.py

stockChecker¥rules¥runner.py

205 lines | 7,614 bytes

```
1  """バックテストルール発見・実行・ランキング計算エンジン
2
3  rules パッケージ内の BaseRule サブクラスを動的に発見し、
4  全銘柄に対するスコアリング・結果保存・総合ランキング算出を統括する。
5  """
6
7  import importlib
8  import inspect
9  import pkgutil
10 from typing import List, Type, Dict
11 from collections.abc import Callable
12
13 from rules.base import BaseRule
14 from db.schema import get_connection, log_error
15
16
17 # importlib / pkgutil / inspect は Python のリフレクション機能 - 実行中にコード自身を調べる
18 # pkgutil.iter_modules でパッケージ内の全 .py ファイルを走査し、BaseRule 継承クラスのみ抽出する
19 # 新しいルールファイルを rules/ に置くだけで自動認識される（設定ファイルの編集不要）
20 def discover_rules() -> List[Type[BaseRule]]:
21     """rules パッケージから BaseRule の具象サブクラスを動的に発見する。
22
23     pkgutil.iter_modules で rules 配下の全モジュールを走査し、
24     BaseRule を継承する具象クラスのみを抽出する（base / runner / __init__ は除外）。
25
26     Returns:
27         List[Type[BaseRule]]: 発見されたルールクラスのリスト。
28
29     使用例:
30         >>> rules = discover_rules()
31         >>> len(rules)
32         8
33
34     注意:
35         - モジュールのインポート順は不定である。
36         - 同一ルールが重複して発見されることはない。
37     """
38     rules = []
39     package = importlib.import_module("rules")
40
41     for importer, modname, ispkg in pkgutil.iter_modules(package.__path__):
42         if modname in ("base", "runner", "__init__"):
43             continue
44         module = importlib.import_module(f"rules.{modname}")
45         for name, obj in inspect.getmembers(module, inspect.isclass):
46             if issubclass(obj, BaseRule) and obj is not BaseRule:
47                 rules.append(obj)
48
49     return rules
50
51
52 def run_backtest(base_date: str) -> Dict[str, List[dict]]:
53     """全ルール・全アクティブ銘柄に対してバックテストを実行する。
54
55     discover_rules() で発見した全ルールを順に実行し、
56     各ルールのスコアリング結果を DB に保存する。
57     最終的に総合ランキングを計算し、結果辞書を返す。
58
59     Args:
60         base_date: 基準日（"YYYY-MM-DD" 形式）。
61
62     Returns:
63         Dict[str, List[dict]]: ルール名をキー、スコアリスト (ticker_id / score) を値とする辞書。
64
65     Raises:
66         なし（個別の calculate 例外は 0.0 スコアで補償され、log_error に記録される）。
67
68     前提条件:
69         - DB に tickers / prices テーブルのデータが存在すること。
70         - discover_rules() で少なくとも 1 つのルールが発見されること。
71     """
```

```

72 rule_classes = discover_rules()
73 rule_instances = [cls() for cls in rule_classes]
74
75 conn = get_connection()
76 cursor = conn.cursor()
77 cursor.execute("""
78     SELECT DISTINCT t.id, t.symbol, t.name
79     FROM tickers t
80     INNER JOIN prices p ON p.ticker_id = t.id
81     WHERE t.is_active = 1
82 """)
83 all_tickers = [dict(r) for r in cursor.fetchall()]
84 if not all_tickers:
85     cursor.execute("SELECT id, symbol, name FROM tickers WHERE is_active = 1")
86     all_tickers = [dict(r) for r in cursor.fetchall()]
87 conn.close()
88
89 results = {}
90
91 for rule in rule_instances:
92     rule_name = rule.name
93     print(f"Running rule: {rule_name}")
94     ticker_scores = []
95
96     # 個別銘柄の計算で例外が発生しても 0.0 で補完して続行 (フォールトトレランス)
97     # エラーは log_error で DB に記録される
98     for ticker in all_tickers:
99         ticker_id = ticker["id"]
100         try:
101             score = rule.calculate(ticker_id, base_date)
102             ticker_scores.append({"ticker_id": ticker_id, "score": score})
103         except Exception as e:
104             ticker_scores.append({"ticker_id": ticker_id, "score": 0.0})
105             log_error(f"rule_{rule_name}", ticker_id, str(e))
106
107     ticker_scores.sort(key=lambda x: x["score"], reverse=True)
108     results[rule_name] = ticker_scores
109
110     save_results(base_date, rule_name, ticker_scores)
111
112 total_results = compute_total_ranking(results, all_tickers)
113 save_results(base_date, "総合ランキング", total_results)
114
115 return results
116
117
118 def compute_total_ranking(
119     rule_results: Dict[str, List[dict]], all_tickers: List[dict]
120 ) -> List[dict]:
121     """全ルールのスコアを合算し、総合ランキング (上位 10 件) を計算する。
122
123     Args:
124         rule_results: ルール名をキー、スコアリストを値とする辞書。
125         all_tickers: 全アクティブ銘柄のリスト (id / symbol / name)。
126
127     Returns:
128         List[dict]: 上位 10 銘柄のランキングリスト。
129         各要素は {"ticker_id": int, "total_score": float, "rule_count": int, "score": float}。
130
131     注意:
132     - スコアが未計算のルールは合算対象から除外される。
133     - 同一銘柄が複数回出現することはない。
134     """
135     ticker_map = {}
136     for t in all_tickers:
137         ticker_map[t["id"]] = {"ticker_id": t["id"], "total_score": 0, "rule_count": 0}
138
139     for rule_name, scores in rule_results.items():
140         for item in scores:
141             tid = item["ticker_id"]
142             if tid in ticker_map:
143                 ticker_map[tid]["total_score"] += item["score"]
144                 ticker_map[tid]["rule_count"] += 1
145
146     rankings = list(ticker_map.values())
147     for r in rankings:
148         if r["rule_count"] > 0:

```

```

149         r["score"] = r["total_score"]
150     else:
151         r["score"] = 0
152
153     rankings.sort(key=lambda x: x["score"], reverse=True)
154     return rankings[:10]
155
156
157 def save_results(base_date: str, rule_name: str, scores: List[dict]) -> None:
158     """バックテスト結果をデータベースの backtest_results テーブルに保存する。
159
160     同一基準日・同一ルールの既存レコードを削除してから INSERT する（冪等性保証）。
161
162     Args:
163         base_date: 基準日（"YYYY-MM-DD" 形式）。
164         rule_name: ルール名称。
165         scores: 保存するスコアリスト。各要素は {"ticker_id": int, "score": float}。
166
167     Returns:
168         None
169
170     Raises:
171         sqlite3.Error: DB 操作に失敗した場合。
172
173     注意:
174     - 順位 (rank) は scores のインデックスから自動採番される (1 始まり)。
175     """
176     conn = get_connection()
177     cursor = conn.cursor()
178
179     cursor.execute("DELETE FROM backtest_results WHERE base_date = ? AND rule_name = ?", (base_date, rule_name))
180
181     for rank, item in enumerate(scores):
182         cursor.execute("""
183             INSERT INTO backtest_results (base_date, rule_name, ticker_id, score, rank)
184             VALUES (?, ?, ?, ?, ?)
185             """, (base_date, rule_name, item["ticker_id"], item["score"], rank + 1))
186
187     conn.commit()
188     conn.close()
189
190
191 def get_rule_names() -> List[str]:
192     """全ルールの名称リストを取得する。
193
194     discover_rules() で発見した各ルールクラスをインスタンス化し、
195     name 属性を収集して返す。
196
197     Returns:
198         List[str]: ルール名称のリスト。
199
200     使用例:
201     >>> get_rule_names()
202     ['モメンタム', 'バリュウ', 'クオリティ', ...]
203     """
204     rules = discover_rules()
205     return [cls.name for cls in rules]

```

sentiment.py

stockChecker¥rules¥sentiment.py

42 lines | 1,625 bytes

```
1 """ニュースセンチメント効果を測定するルール
2
3 事前にダウンロード・分析されたセンチメントスコアを indicators テーブルから読み取る。
4 """
5
6 from rules.base import BaseRule
7
8
9 class SentimentRule(BaseRule):
10     """ニュースセンチメント効果ルール
11
12     indicators テーブルから FinBERT による事前計算済みのセンチメントスコアを読み取る。
13     """
14
15     name = "センチメント"
16     description = "ポジティブニュースが増えると短期的に株価が上がりやすい"
17
18     def need_financials(self) -> bool:
19         """センチメントは indicators テーブルから取得するため、財務データは不要。
20
21         Returns:
22             bool: 常に False。
23         """
24         return False
25
26     # このルールはスコア計算を外部 (downloader_sentiment.py) に委譲している
27     # 自分の役割は indicators テーブルから既存の結果を読み取ることだけ
28     def calculate(self, ticker_id: int, base_date: str) -> float:
29         """事前計算されたセンチメントスコアを indicators テーブルから読み取る。
30
31         Args:
32             ticker_id: 評価対象の銘柄 ID。
33             base_date: 基準日 ("YYYY-MM-DD" 形式)。
34
35         Returns:
36             float: 0~100 のセンチメントスコア。
37                 未計算の場合は 40.0 (中立よりやや低め) を返す。
38         """
39         score = self.get_indicator(ticker_id, base_date, "sentiment")
40         if score is None:
41             return 40.0
42         return score
```

size.py

stockChecker¥rules¥size.py

73 lines | 2,494 bytes

```
1  """サイズ効果（小型株が大型株をアウトパフォームする傾向）を測定するルール
2
3  時価総額に基づいて小型株ほど高スコアをつける。
4  """
5
6  from rules.base import BaseRule
7  from db.schema import get_connection
8
9
10 class SizeRule(BaseRule):
11     """サイズ効果ルール
12
13     時価総額が小さい銘柄ほど高スコアを割り当てる。
14     """
15
16     name = "サイズ"
17     description = "小型株は大型株より長期的に高いリターンを出しやすい"
18
19     SMALL_CAP_THRESHOLD = 2e9
20     MID_CAP_THRESHOLD = 1e10
21     LARGE_CAP_THRESHOLD = 5e10
22     MEGA_CAP_THRESHOLD = 2e11
23
24     def need_financials(self) -> bool:
25         """時価総額のみで判定するため、財務データは不要。
26
27         Returns:
28             bool: 常に False。
29         """
30         return False
31
32     def calculate(self, ticker_id: int, base_date: str) -> float:
33         """時価総額に基づいてサイズスコアを計算する。
34
35         Args:
36             ticker_id: 評価対象の銘柄 ID。
37             base_date: 基準日（"YYYY-MM-DD" 形式、Size ルールでは未使用）。
38
39         Returns:
40             float: 0~100 のスコア。
41                 - 小型（~20億未満）: 85.0
42                 - 中小型（~100億未満）: 70.0
43                 - 中型（~500億未満）: 55.0
44                 - 大型（~2000億未満）: 40.0
45                 - 超大型（2000億以上）: 25.0
46                 時価総額不明の場合は 50.0。
47
48         前提条件:
49             - tickers テーブルの market_cap に値が設定されていること。
50         """
51         conn = get_connection()
52         cursor = conn.cursor()
53         cursor.execute("SELECT market_cap FROM tickers WHERE id = ?", (ticker_id,))
54         row = cursor.fetchone()
55         conn.close()
56
57         mcap = row["market_cap"] if row else None
58
59         if mcap is None or mcap <= 0:
60             return 50.0
61
62         # 閾値 (threshold) で複数のカテゴリに分割した判断基準（ルールベース）
63         # 時価総額が小さいほど高スコア（小型株効果の理論に基づく）
64         if mcap < self.SMALL_CAP_THRESHOLD:
65             return 85.0
66         elif mcap < self.MID_CAP_THRESHOLD:
67             return 70.0
68         elif mcap < self.LARGE_CAP_THRESHOLD:
69             return 55.0
70         elif mcap < self.MEGA_CAP_THRESHOLD:
71             return 40.0
```



```
72     else:
73         return 25.0
```

text_score.py

stockChecker¥rules¥text_score.py

60 lines | 2,191 bytes

```
1 """テキスト特徴（決算書の楽観性）を測定するルール
2
3 事前計算されたテキストスコアを indicators テーブルから読み取り、
4 フォールバックとしてROEベースの簡易スコアを返す。
5 """
6
7 from rules.base import BaseRule
8
9
10 class TextScoreRule(BaseRule):
11     """テキスト特徴ルール
12
13     事前計算されたテキストスコア（決算書の楽観性）を indicators テーブルから取得する。
14     フォールバックとして ROE ベースの簡易スコアを返す。
15     """
16
17     name = "テキスト特徴"
18     description = "決算書の文章が「楽観的」な企業は、その後の株価が上がりやすい"
19
20     def need_financials(self) -> bool:
21         """フォールバックで ROE を参照するため、財務データは必須。
22
23         Returns:
24             bool: 常に True。
25         """
26         return True
27
28     # フォールバック（fallback）：本来のデータが存在しない場合の代替処理
29     # テキスト分析結果があればそれを優先し、なければ ROE で簡易推定する
30     def calculate(self, ticker_id: int, base_date: str) -> float:
31         """テキストスコアまたは ROE ベースのフォールバックスコアを計算する。
32
33         Args:
34             ticker_id: 評価対象の銘柄 ID。
35             base_date: 基準日（"YYYY-MM-DD" 形式）。
36
37         Returns:
38             float: 0~100 のスコア。
39                 事前計算済みテキストスコア優先、無い場合は ROE から推定。
40                 データが一切ない場合は 40.0。
41
42         注意:
43             - ROE フォールバック: ROE > 10% -> 65.0, > 5% -> 55.0, それ以下 -> 45.0
44         """
45         score = self.get_indicator(ticker_id, base_date, "text_score")
46         if score is not None:
47             return score
48
49         fin = self.get_latest_financial(ticker_id, base_date)
50         if fin is None:
51             return 40.0
52
53         roe = fin.get("roe")
54
55         if roe is not None and roe > 0.1:
56             return 65.0
57         if roe is not None and roe > 0.05:
58             return 55.0
59
60         return 45.0
```

value.py

stockChecker¥rules¥value.py

74 lines | 2,629 bytes

```
1 """バリュウ効果（割安株が市場平均を上回る傾向）を測定するルール
2
3 PER・PBR・配当利回りを組み合わせて総合的な割安度をスコア化する。
4 """
5
6 from rules.base import BaseRule
7
8
9 class ValueRule(BaseRule):
10     """バリュウ効果ルール
11
12     PER・PBR・配当利回りから総合的な割安度をスコアリングする。
13     """
14
15     name = "バリュウ"
16     description = "割安株（PBR・PERが低い）は長期的に市場平均を上回りやすい"
17
18     def need_financials(self) -> bool:
19         """PER・PBR・配当利回りを参照するため、財務データは必須。
20
21         Returns:
22             bool: 常に True。
23         """
24         return True
25
26     def calculate(self, ticker_id: int, base_date: str) -> float:
27         """PER・PBR・配当利回りを加重平均して割安度スコアを計算する。
28
29         Args:
30             ticker_id: 評価対象の銘柄 ID。
31             base_date: 基準日（"YYYY-MM-DD" 形式）。
32
33         Returns:
34             float: 0~100 のスコア。値が大きいほど割安と判定される。
35                 財務データがない場合は 40.0 を返す。
36
37         注意:
38             - PER は 40%、PBR は 40%、配当利回りは 20% の比率で加重。
39         """
40         fin = self.get_latest_financial(ticker_id, base_date)
41         if fin is None:
42             return 40.0
43
44         per = fin.get("per")
45         pbr = fin.get("pbr")
46         div_yield = fin.get("dividend_yield")
47
48         score = 0
49         weights = 0
50
51         # PER と PBR は「低いほど割安」→ 100-per*2 で逆転スコアに
52         # 配当利回りは「高いほど良い」（ただし日本の低位株は注意）
53         # max(0, min(100, ...)) でスコアを 0~100 にクリッピング（はみ出し防止）
54         if per is not None and per > 0:
55             per_score = max(0, min(100, 100 - per * 2))
56             score += per_score * 0.4
57             weights += 0.4
58
59         if pbr is not None and pbr > 0:
60             pbr_score = max(0, min(100, 100 - pbr * 20))
61             score += pbr_score * 0.4
62             weights += 0.4
63
64         if div_yield is not None and div_yield > 0:
65             div_score = min(100, div_yield * 1000)
66             score += div_score * 0.2
67             weights += 0.2
68
69         # どの指標も取得できなかった場合のデフォルト値（中央より少し低め）
70         if weights == 0:
71             return 40.0
```

```
72
73 # weights で割って加重平均を取る（実際に使えた指標のみで平均）
74 return score / weights
```

seed.py

stockChecker¥seed.py
108 lines | 4,104 bytes

```
1  """stockChecker 初期データ投入スクリプト
2
3  指定された10銘柄（AAPL, MSFT 等）の株価・財務データを yfinance からダウンロードし、
4  すぐにバックテストを実行できる状態にデータベースを初期化する。
5
6  使用例:
7      python seed.py
8  """
9
10 import time
11 import yfinance as yf
12 from db.schema import init_db, get_connection
13 from db.tickers import update_ticker_list
14 from db.downloader_prices import download_single_price, save_prices
15 from db.downloader_financials import fetch_yfinance_financials, extract_fiscal_year_data, save_financials
16
17 # すぐにバックテストを試すための厳選 10 銘柄
18 DEMO_SYMBOLS = ["AAPL", "MSFT", "GOOGL", "AMZN", "NVDA", "META", "TSLA", "JPM", "V", "KO"]
19
20
21 def seed():
22     """デモ用の 10 銘柄データをダウンロードし、データベースに保存する。
23
24     銘柄ごとに時価総額・株価（10年分）・財務データを順次取得し、
25     データベースの tickers / prices / financials テーブルへ保存する。
26
27     Args:
28         なし
29
30     Returns:
31         None
32
33     Raises:
34         KeyboardInterrupt: Ctrl+C による中断。
35         その他の例外は個別にキャッチされ、警告メッセージが表示される。
36
37     注意:
38         - 各銘柄間に 0.5 秒のスリープを入れ、API 負荷を軽減する。
39         - すでに存在する銘柄はスキップされず、時価総額のみ更新される。
40     """
41     print("=== Initializing database ===")
42     init_db()
43
44     print(f"\n=== Updating ticker list ===")
45     update_ticker_list()
46
47     print(f"\n=== Seeding demo data (10 US stocks) ===")
48     conn = get_connection()
49     cursor = conn.cursor()
50
51     # DEMO_SYMBOLS の各銘柄について、時価総額・株価・財務データを順に取得する
52     for symbol in DEMO_SYMBOLS:
53         # ? は SQL インジェクション対策のプレースホルダ。タプル (symbol,) で値をバインドする
54         cursor.execute("SELECT id FROM tickers WHERE symbol = ?", (symbol,))
55         row = cursor.fetchone()
56         if not row:
57             cursor.execute(
58                 "INSERT INTO tickers (symbol, name, market) VALUES (?, ?, 'us')",
59                 (symbol, symbol),
60             )
61             conn.commit()
62             ticker_id = cursor.lastrowid
63         else:
64             ticker_id = row["id"]
65
66     print(f"\n[{symbol}] Fetching market cap...")
67     # yf.Ticker は yfinance ライブラリの中心クラス。銘柄シンボルを渡して株価・財務情報を取得する
68     try:
69         tk = yf.Ticker(symbol)
70         info = tk.info
71         mcap = info.get("marketCap")
```

```

72         if mcap:
73             cursor.execute("UPDATE tickers SET market_cap = ?, updated_at = datetime('now') WHERE id = ?",
74                             (mcap, ticker_id))
75             conn.commit()
76             print(f" -> market_cap={mcap}")
77     except Exception as e:
78         print(f" -> could not fetch market cap: {e}")
79
80     print(f"[{symbol}] Downloading prices (10 years)...")
81     df = download_single_price(symbol, "2015-01-01", "2026-07-09")
82     if df is not None and not df.empty:
83         save_prices(ticker_id, df)
84         print(f" -> {len(df)} price rows saved")
85     else:
86         print(" -> No price data")
87
88     print(f"[{symbol}] Downloading financials...")
89     raw = fetch_yfinance_financials(symbol)
90     if raw is not None:
91         records = extract_fiscal_year_data(raw)
92         if records:
93             save_financials(ticker_id, records)
94             print(f" -> {len(records)} fiscal years saved")
95     else:
96         print(" -> No financial data")
97
98     # 連続リクエストで API 制限にかからないよう、銘柄間に 0.5 秒のインターバルを入れる
99     time.sleep(0.5)
100
101     conn.close()
102     print("\n=== Seed complete! ===")
103     print("Run: streamlit run app.py")
104     print("Then click 'バックテスト実行' to see results.")
105
106
107 if __name__ == "__main__":
108     seed()

```

`__init__.py`

stockChecker¥tests¥__init__.py

1 lines | 0 bytes

1

test_rules.py

stockChecker¥tests¥test_rules.py

156 lines | 5,713 bytes

```
1 """各ルールのスコアリングとルール発見機構のテスト
2
3 各ルールが0~100の範囲のスコアを返すこと、および全9ルールが自動発見されることを検証する。
4 """
5
6 import os
7 import pytest
8 import tempfile
9
10 from db.schema import init_db, get_connection
11
12
13 @pytest.fixture(autouse=True)
14 def temp_db(monkeypatch):
15     """テスト用の一時データベースを作成・初期化し、テストデータを投入するフィクスチャ。
16
17     config.DB_PATH を一時ファイルに差し替え、tickers (2 銘柄) ・prices (12 ヶ月分) ・
18     financials (1 件) のテストデータを投入する。
19
20     Yields:
21         None
22     """
23     with tempfile.NamedTemporaryFile(suffix=".db", delete=False) as f:
24         db_path = f.name
25         monkeypatch.setattr("config.DB_PATH", db_path)
26         init_db()
27
28         conn = get_connection()
29         cursor = conn.cursor()
30         cursor.execute("INSERT INTO tickers (symbol, name, market) VALUES (?, ?, ?)", ("AAPL", "Apple", "us"))
31         cursor.execute("INSERT INTO tickers (symbol, name, market) VALUES (?, ?, ?)", ("7203.T", "Toyota", "japan"))
32         # cursor.executemany は同じ SQL を複数のパラメータで繰り返し実行する
33         # リスト内包表記 [式 for i, m in enumerate(range(1,13))] で 12 ヶ月分のデータを生成
34         cursor.executemany(
35             "INSERT INTO prices (ticker_id, date, close, volume) VALUES (?, ?, ?, ?)",
36             [
37                 (1, f"2023-{m:02d}-01", 150 + i * 2, 1000000 + i * 10000)
38                 for i, m in enumerate(range(1, 13))
39             ],
40         )
41         cursor.execute("""
42             INSERT INTO financials (ticker_id, fiscal_year, revenue, operating_income, net_income,
43                                     total_assets, total_equity, per, pbr, roe, dividend_yield)
44             VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
45             """, (1, 2023, 400000, 120000, 95000, 350000, 150000, 25, 40, 0.25, 0.005))
46         conn.commit()
47         conn.close()
48         yield
49         os.unlink(db_path)
50
51
52 def test_momentum_rule():
53     """MomentumRule が 0~100 のスコアを返すことを検証する。"""
54     from rules.momentum import MomentumRule
55     rule = MomentumRule()
56     score = rule.calculate(1, "2024-12-31")
57     assert 0 <= score <= 100
58
59
60 def test_value_rule():
61     """ValueRule が 0~100 のスコアを返すことを検証する。"""
62     from rules.value import ValueRule
63     rule = ValueRule()
64     score = rule.calculate(1, "2024-12-31")
65     assert 0 <= score <= 100
66
67
68 def test_quality_rule():
69     """QualityRule が 0~100 のスコアを返すことを検証する。"""
70     from rules.quality import QualityRule
71     rule = QualityRule()
```



```

72     score = rule.calculate(1, "2024-12-31")
73     assert 0 <= score <= 100
74
75
76 def test_low_vol_rule():
77     """LowVolRule が 0~100 のスコアを返すことを検証する。"""
78     from rules.low_vol import LowVolRule
79     rule = LowVolRule()
80     score = rule.calculate(1, "2024-12-31")
81     assert 0 <= score <= 100
82
83
84 def test_size_rule():
85     """SizeRule が 0~100 のスコアを返すことを検証する。"""
86     from rules.size import SizeRule
87     rule = SizeRule()
88     score = rule.calculate(1, "2024-12-31")
89     assert 0 <= score <= 100
90
91
92 def test_anomaly_rule():
93     """AnomalyRule が 0~100 のスコアを返すことを検証する。"""
94     from rules.anomaly import AnomalyRule
95     rule = AnomalyRule()
96     score = rule.calculate(1, "2024-12-31")
97     assert 0 <= score <= 100
98
99
100 def test_text_score_rule():
101     """TextScoreRule が 0~100 のスコアを返すことを検証する。"""
102     from rules.text_score import TextScoreRule
103     rule = TextScoreRule()
104     score = rule.calculate(1, "2024-12-31")
105     assert 0 <= score <= 100
106
107
108 def test_sentiment_rule():
109     """SentimentRule が 0~100 のスコアを返すことを検証する。"""
110     from rules.sentiment import SentimentRule
111     rule = SentimentRule()
112     score = rule.calculate(1, "2024-12-31")
113     assert 0 <= score <= 100
114
115
116 def test_pead_rule():
117     """PeadRule が 0~100 のスコアを返すことを検証する。"""
118     from rules.pead import PeadRule
119     rule = PeadRule()
120     score = rule.calculate(1, "2024-12-31")
121     assert 0 <= score <= 100
122
123
124 def test_discover_rules():
125     """discover_rules() が全 9 ルールを発見することを検証する。"""
126     from rules.runner import discover_rules
127     rules = discover_rules()
128     rule_names = [r.name for r in rules]
129     expected = ["モメンタム", "バリュー", "クオリティ", "サイズ", "低ボラティリティ", "PEAD", "センチメント", "テキスト特徴", "異常検知"...
130     for name in expected:
131         assert name in rule_names, f"Missing rule: {name}"
132
133
134 def test_runner_backtest():
135     """run_backtest() が全ルールスコアを計算し、0~100 の範囲であることを検証する。"""
136     from rules.runner import run_backtest
137     results = run_backtest("2024-12-31")
138     assert len(results) >= 8
139     for rule_name, scores in results.items():
140         assert len(scores) > 0
141         for item in scores:
142             assert "ticker_id" in item
143             assert "score" in item
144             assert 0 <= item["score"] <= 100
145
146
147 def test_ranking_build():
148     """build_total_ranking_simple() がスコア降順のランキングを生成することを検証する。"""

```

```
149 from ui.ranking import build_total_ranking_simple
150 mock_results = {
151     "モメンタム": [{"ticker_id": 1, "score": 80}, {"ticker_id": 2, "score": 60}],
152     "バリュー": [{"ticker_id": 1, "score": 70}, {"ticker_id": 2, "score": 90}],
153 }
154 ranking = build_total_ranking_simple(mock_results)
155 assert len(ranking) == 2
156 assert ranking[0]["score"] >= ranking[1]["score"]
```

test_schema.py

stockChecker¥tests¥test_schema.py

112 lines | 3,772 bytes

```
1 """DBスキーマのテスト
2
3 テーブル作成・Ticker CRUD・価格データのUPSERTを検証する。
4 """
5
6 import os
7 import pytest
8 import tempfile
9
10 from db.schema import init_db, get_connection
11
12
13 # @pytest.fixture はテストの「準備→実行→後片付け」を定義するデコレータ
14 # autouse=True で全テストに自動適用（各テスト関数が個別に呼び出す必要なし）
15 # monkeypatch.setattr で config.DB_PATH を一時ファイルに差し替え（テスト間の DB 分離）
16 # yield の前 = 準備（setup）、後 = 後片付け（teardown）。os.unlink で一時ファイル削除
17 @pytest.fixture(autouse=True)
18 def temp_db(monkeypatch):
19     """テスト用の一時データベースを作成するフィクスチャ。
20
21     config.DB_PATH を一時ファイルに差し替え、テスト後に自動削除する。
22
23     Yields:
24         None
25     """
26     with tempfile.NamedTemporaryFile(suffix=".db", delete=False) as f:
27         db_path = f.name
28         monkeypatch.setattr("config.DB_PATH", db_path)
29         yield
30         os.unlink(db_path)
31
32
33 def test_init_db_creates_tables():
34     """init_db() が全テーブルを作成することを確認する。"""
35     init_db()
36     conn = get_connection()
37     cursor = conn.cursor()
38     cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
39     tables = {row["name"] for row in cursor.fetchall()}
40     conn.close()
41
42     expected = {
43         "tickers", "prices", "financials", "indicators",
44         "backtest_results", "sentiment_download_log", "error_log",
45     }
46     assert expected.issubset(tables), f"Missing tables: {expected - tables}"
47
48
49 def test_ticker_insert():
50     """tickers テーブルへの INSERT および SELECT を検証する。"""
51     init_db()
52     conn = get_connection()
53     cursor = conn.cursor()
54     cursor.execute(
55         "INSERT INTO tickers (symbol, name, market) VALUES (?, ?, ?)",
56         ("AAPL", "Apple Inc.", "us"),
57     )
58     conn.commit()
59
60     cursor.execute("SELECT * FROM tickers WHERE symbol = 'AAPL'")
61     row = cursor.fetchone()
62     conn.close()
63
64     assert row is not None
65     assert row["name"] == "Apple Inc."
66     assert row["market"] == "us"
67
68
69 def test_prices_insert_and_duplicate():
70     """prices テーブルの INSERT および INSERT OR REPLACE (UPSERT) を検証する。"""
71     init_db()
```

```

72     conn = get_connection()
73     cursor = conn.cursor()
74     cursor.execute("INSERT INTO tickers (symbol, name, market) VALUES (?, ?, ?)", ("MSFT", "Microsoft", "us"))
75     conn.commit()
76
77     cursor.execute("INSERT INTO prices (ticker_id, date, close, volume) VALUES (?, ?, ?, ?)",
78                   (1, "2024-01-05", 155.0, 1000000))
79     conn.commit()
80
81     cursor.execute("INSERT OR REPLACE INTO prices (ticker_id, date, close, volume) VALUES (?, ?, ?, ?)",
82                   (1, "2024-01-05", 155.0, 2000000))
83     conn.commit()
84
85     cursor.execute("SELECT close, volume FROM prices WHERE ticker_id = 1 AND date = '2024-01-05'")
86     row = cursor.fetchone()
87     conn.close()
88
89     assert row["close"] == 155.0
90     assert row["volume"] == 2000000
91
92
93 def test_log_error():
94     """log_error() が error_log テーブルにレコードを作成することを確認する。"""
95     init_db()
96     conn = get_connection()
97     cursor = conn.cursor()
98     cursor.execute("INSERT INTO tickers (symbol, name, market) VALUES (?, ?, ?)", ("TEST", "Test Inc.", "us"))
99     conn.commit()
100    conn.close()
101
102    from db.schema import log_error
103    log_error("test_operation", 1, "test error message")
104
105    conn = get_connection()
106    cursor = conn.cursor()
107    cursor.execute("SELECT * FROM error_log WHERE operation = 'test_operation'")
108    row = cursor.fetchone()
109    conn.close()
110
111    assert row is not None
112    assert row["error_message"] == "test error message"

```

`__init__.py`

stockChecker¥ui¥__init__.py

1 lines | 0 bytes

1

charts.py

stockChecker¥ui¥charts.py

134 lines | 4,549 bytes

```
1 """Plotlyを使用した株価チャート描画モジュール
2
3 ローソク足チャート・20日移動平均線・出来高を描画する。
4 """
5
6 import plotly.graph_objects as go
7 import pandas as pd
8 from datetime import datetime, timedelta
9 from typing import Optional
10
11 from db.schema import get_connection
12
13
14 def fetch_prices_for_chart(ticker_id: int, base_date: str, months: int) -> Optional[pd.DataFrame]:
15     """チャート描画用の株価データを DB から取得する。
16
17     Args:
18         ticker_id: 銘柄 ID。
19         base_date: 基準日 ("YYYY-MM-DD" 形式)。
20         months: 取得月数 (例: 6 → 6 ヶ月前から基準日まで)。
21
22     Returns:
23         Optional[pd.DataFrame]: date / open / high / low / close / volume カラムの DataFrame。
24         データが存在しない場合は None。
25     """
26     conn = get_connection()
27     end = datetime.strptime(base_date, "%Y-%m-%d")
28     start = end - timedelta(days=months * 30)
29
30     df = pd.read_sql_query("""
31         SELECT date, open, high, low, close, volume
32         FROM prices
33         WHERE ticker_id = ? AND date >= ? AND date <= ?
34         ORDER BY date ASC
35     """, conn, params=(ticker_id, start.strftime("%Y-%m-%d"), base_date))
36     conn.close()
37
38     if df.empty:
39         return None
40     df["date"] = pd.to_datetime(df["date"])
41     return df
42
43
44 def create_price_chart(ticker_id: int, base_date: str, months: int, ticker_name: str = "") -> go.Figure:
45     """Plotly のローソク足チャートを作成する。
46
47     ローソク足・20 日移動平均線・出来高バーを重ねて表示する。
48     OHLC データが利用できない場合は折れ線グラフで代替する。
49
50     Args:
51         ticker_id: 銘柄 ID。
52         base_date: 基準日 ("YYYY-MM-DD" 形式)。
53         months: 表示期間 (月数)。
54         ticker_name: チャートタイトルに表示する銘柄名。
55
56     Returns:
57         go.Figure: Plotly Figure オブジェクト。データがなくても空の図を返す。
58     """
59     df = fetch_prices_for_chart(ticker_id, base_date, months)
60
61     fig = go.Figure()
62
63     if df is None or df.empty:
64         fig.add_annotation(
65             text="株価データがありません (銘柄リスト更新→株価DLを実行してください)",
66             showarrow=False,
67             font=dict(size=14),
68         )
69     fig.update_layout(height=300)
70     return fig
71
```

```

72 has_ohlc = all(c in df.columns and df[c].notna().any() for c in ["open", "high", "low", "close"])
73
74 # go.Candlestick は Plotly のローソク足チャート。4 本値（始値・高値・安値・終値）を指定
75 # OHLC (Open/High/Low/Close) データがあればローソク足、なければ折れ線グラフ
76 if has_ohlc:
77     fig.add_trace(go.Candlestick(
78         x=df["date"],
79         open=df["open"],
80         high=df["high"],
81         low=df["low"],
82         close=df["close"],
83         name="Price",
84         showlegend=False,
85     ))
86 else:
87     fig.add_trace(go.Scatter(
88         x=df["date"],
89         y=df["close"] if "close" in df.columns else df.iloc[:, 0],
90         mode="lines",
91         name="Close",
92         line=dict(color="#00b4d8", width=2),
93     ))
94
95 # rolling(20).mean() は 20 日間の移動平均（窓関数）。株価チャートの定番テクニカル指標
96 if len(df) >= 20 and "close" in df.columns:
97     ma20 = df["close"].rolling(20).mean()
98     fig.add_trace(go.Scatter(
99         x=df["date"],
100         y=ma20,
101         mode="lines",
102         name="20MA",
103         line=dict(color="orange", width=1),
104     ))
105
106 if "volume" in df.columns and df["volume"].notna().any():
107     vol_colors = ["gray"] * len(df)
108     fig.add_trace(go.Bar(
109         x=df["date"],
110         y=df["volume"],
111         name="出来高",
112         yaxis="y2",
113         marker=dict(color="rgba(100, 100, 100, 0.3)"),
114         showlegend=True,
115     ))
116
117 title = f"{ticker_name} - {months}ヶ月" if ticker_name else f"{months}ヶ月"
118 fig.update_layout(
119     title=title,
120     height=350,
121     margin=dict(l=20, r=20, t=40, b=20),
122     xaxis_rangeslider_visible=False,
123     template="plotly_white",
124     yaxis=dict(title="株価"),
125     yaxis2=dict(
126         title="出来高",
127         overlaying="y",
128         side="right",
129         showgrid=False,
130     ),
131     legend=dict(x=0, y=1.1, orientation="h"),
132 )
133
134 return fig

```

ranking.py

stockChecker¥ui¥ranking.py

81 lines | 3,037 bytes

```
1 """ランキングデータ構築モジュール
2
3 各ルールのスコアリング結果を統合し、総合ランキングデータを生成する。
4 """
5
6 from typing import Dict, List
7
8
9 # 辞書に銘柄ごとのスコアを累積 → スコア降順にソート → ランキング
10 # このパターン（集約→ソート）はランキング生成の基本的なアルゴリズム
11 def build_ranking_data(
12     rule_results: Dict[str, List[dict]]
13 ) -> List[dict]:
14     """全ルールのスコアを銘柄ごとに集約し、総合スコア順にソートされたランキングを生成する。
15
16     Args:
17         rule_results: ルール名をキー、スコアリストを値とする辞書。
18
19     Returns:
20         List[dict]: 各要素が {"ticker_id": int, "total_score": float, "rules": {rule_name: score}} のリスト。
21                     総合スコア降順。
22
23     使用例:
24     >>> build_ranking_data({"モメンタム": [{"ticker_id": 1, "score": 80}]}
25         [{"ticker_id": 1, "total_score": 80, "rules": {"モメンタム": 80}}]
26     """
27     ticker_scores = {}
28
29     for rule_name, scores in rule_results.items():
30         for item in scores:
31             tid = item["ticker_id"]
32             if tid not in ticker_scores:
33                 ticker_scores[tid] = {
34                     "ticker_id": tid,
35                     "total_score": 0,
36                     "rules": {},
37                 }
38             ticker_scores[tid]["total_score"] += item["score"]
39             ticker_scores[tid]["rules"][rule_name] = item["score"]
40
41     ranking = list(ticker_scores.values())
42     ranking.sort(key=lambda x: x["total_score"], reverse=True)
43     return ranking
44
45
46 def build_total_ranking_simple(results: Dict[str, List[dict]]) -> List[dict]:
47     """シンプルな総合ランキングを生成する（ルール別内訳なし）。
48
49     build_ranking_data の軽量化版。総合スコアのみを保持したランキングリストを返す。
50
51     Args:
52         results: ルール名をキー、スコアリストを値とする辞書。
53
54     Returns:
55         List[dict]: 各要素が {"ticker_id": int, "score": float} のリスト。
56                     総合スコア降順。
57
58     使用例:
59     >>> build_total_ranking_simple({"モメンタム": [{"ticker_id": 1, "score": 80}]}
60         [{"ticker_id": 1, "score": 80}]
61     """
62     ticker_scores = {}
63     for rule_name, scores in results.items():
64         for item in scores:
65             tid = item["ticker_id"]
66             if tid not in ticker_scores:
67                 ticker_scores[tid] = {"ticker_id": tid, "total_score": 0, "rules": {}}
68             ticker_scores[tid]["total_score"] += item["score"]
69             ticker_scores[tid]["rules"][rule_name] = item["score"]
70
71     ranking = list(ticker_scores.values())
```



```
72 ranking.sort(key=lambda x: x["total_score"], reverse=True)
73
74 result_list = []
75 for r in ranking:
76     result_list.append({
77         "ticker_id": r["ticker_id"],
78         "score": r["total_score"],
79     })
80
81 return result_list
```

tabs.py

stockChecker¥ui¥tabs.py

148 lines | 5,339 bytes

```
1 """Streamlit UI のタブ描画モジュール
2
3 各ルールごとの結果タブと総合ランキングタブをレンダリングする。
4 """
5
6 import streamlit as st
7 from typing import Dict, List
8 from datetime import datetime
9
10 from ui.charts import create_price_chart
11 from db.schema import get_connection
12
13
14 MONTH_OPTIONS = [3, 6, 9, 12]
15
16
17 # ST は Streamlit の標準的な別名。st.* がすべての UI 命令を提供する
18 # st.columns([1, 3, 1]) は画面を 1:3:1 の比率で横分割するレイアウト機能
19 # st.expander("タイトル") は折りたたみ可能なセクションを作成する
20 # st.radio(... horizontal=True) で水平ラジオボタン (デフォルトは縦)
21 def render_tab(rule_name: str, scores: list, base_date: str) -> None:
22     """ルール別の結果タブをレンダリングする。
23
24     スコア上位 50 銘柄をランキング表示し、各銘柄に株価チャートの展開可能セクションを設置する。
25
26     Args:
27         rule_name: ルール名称。
28         scores: スコアリスト (ticker_id / score) 。
29         base_date: 基準日 ("YYYY-MM-DD" 形式) 。
30
31     Returns:
32         None
33     """
34     ticker_map = _get_ticker_map()
35
36     if not scores:
37         st.info("該当する銘柄がありません")
38         return
39
40     for rank, item in enumerate(scores[:50]):
41         ticker_id = item["ticker_id"]
42         score = item["score"]
43         ticker = ticker_map.get(ticker_id, {})
44
45         symbol = ticker.get("symbol", f"ID:{ticker_id}")
46         name = ticker.get("name", "")
47
48         col1, col2, col3 = st.columns([1, 3, 1])
49         with col1:
50             st.write(f"##[{rank+1}]")
51         with col2:
52             st.write(f"{name} ({symbol})")
53         with col3:
54             st.write(f"Score: **{score:.1f}**")
55
56         expand = st.expander(f"グラフ / {symbol}", expanded=True)
57         with expand:
58             render_chart_controls(ticker_id, symbol, base_date, rule_name)
59
60
61 def render_total_tab(total_results: list, base_date: str, rule_scores: Dict[str, list]) -> None:
62     """総合ランキングタブ (TOP10) をレンダリングする。
63
64     各銘柄の総合スコアとルール別スコアを表示し、株価チャートの展開可能セクションを設置する。
65
66     Args:
67         total_results: 総合ランキングリスト (ticker_id / score) 。
68         base_date: 基準日 ("YYYY-MM-DD" 形式) 。
69         rule_scores: ルール別スコア辞書。
70
71     Returns:
```

```

72     None
73     """
74     ticker_map = _get_ticker_map()
75
76     if not total_results:
77         st.info("総合ランキングを計算できませんでした")
78         return
79
80     st.subheader(f"総合ランキング TOP10 (基準日: {base_date})")
81
82     for rank, item in enumerate(total_results[:10]):
83         ticker_id = item["ticker_id"]
84         total_score = item["score"]
85         ticker = ticker_map.get(ticker_id, {})
86         symbol = ticker.get("symbol", f"ID:{ticker_id}")
87         name = ticker.get("name", "")
88
89         individual_scores = {}
90         for rule_name, scores in rule_scores.items():
91             for si in scores:
92                 if si["ticker_id"] == ticker_id:
93                     individual_scores[rule_name] = si["score"]
94                     break
95
96         st.markdown(f"### #{rank+1} {name} ({symbol}) - 総合: {total_score:.1f}")
97
98         score_cols = st.columns(min(5, len(individual_scores)))
99         for idx, (rname, rscore) in enumerate(list(individual_scores.items())[:5]):
100             with score_cols[idx]:
101                 st.metric(rname, f"{rscore:.1f}")
102
103         expand = st.expander(f"グラフ / {symbol}", expanded=True)
104         with expand:
105             render_chart_controls(ticker_id, symbol, base_date, "総合")
106
107         st.divider()
108
109
110 def render_chart_controls(ticker_id: int, symbol: str, base_date: str, tab_label: str = "") -> None:
111     """株価チャートの期間選択ラジオボタンとチャート描画を制御する。
112
113     Args:
114         ticker_id: 銘柄 ID。
115         symbol: 銘柄シンボル。
116         base_date: 基準日 ("YYYY-MM-DD" 形式)。
117         tab_label: タブ識別ラベル (キー重複防止用)。
118
119     Returns:
120         None
121     """
122     key_base = f"chart_{tab_label}_{ticker_id}_{symbol}"
123
124     months = st.radio(
125         "期間", MONTH_OPTIONS,
126         index=MONTH_OPTIONS.index(6),
127         horizontal=True,
128         key=f"{key_base}_radio",
129     )
130
131     fig = create_price_chart(ticker_id, base_date, months, symbol)
132     st.plotly_chart(fig, key=f"{key_base}_chart")
133
134
135 # 名前が _ で始まる関数は「内部使用 (プライベート)」の慣習 - 外部から呼ばれる想定ではない
136 def _get_ticker_map() -> Dict[int, dict]:
137     """全銘柄の ID → (symbol / name) マッピング辞書を取得する (内部ヘルパー)。
138
139     Returns:
140         Dict[int, dict]: ticker_id をキー、{"symbol": str, "name": str} を値とする辞書。
141     """
142     conn = get_connection()
143     cursor = conn.cursor()
144     cursor.execute("SELECT id, symbol, name FROM tickers")
145     # 辞書内包表記: {キー: 値 for 要素 in イテラブル} - ループで辞書を組み立てる Python の簡潔記法
146     result = {row["id"]: {"symbol": row["symbol"], "name": row["name"]} for row in cursor.fetchall()}
147     conn.close()
148     return result

```

File Index

stockChecker¥app.py	3-4
stockChecker¥config.py	5
stockChecker¥db¥__init__.py	6
stockChecker¥db¥downloader_financials.py	7-10
stockChecker¥db¥downloader_prices.py	11-13
stockChecker¥db¥downloader_sentiment.py	14-17
stockChecker¥db¥schema.py	18-20
stockChecker¥db¥tickers.py	21-23
stockChecker¥rules¥__init__.py	24
stockChecker¥rules¥anomaly.py	25-26
stockChecker¥rules¥base.py	27-29
stockChecker¥rules¥low_vol.py	30
stockChecker¥rules¥momentum.py	31
stockChecker¥rules¥pead.py	32-33
stockChecker¥rules¥quality.py	34-35
stockChecker¥rules¥runner.py	36-38
stockChecker¥rules¥sentiment.py	39
stockChecker¥rules¥size.py	40-41
stockChecker¥rules¥text_score.py	42
stockChecker¥rules¥value.py	43-44
stockChecker¥seed.py	45-46
stockChecker¥tests¥__init__.py	47
stockChecker¥tests¥test_rules.py	48-50
stockChecker¥tests¥test_schema.py	51-52
stockChecker¥ui¥__init__.py	53
stockChecker¥ui¥charts.py	54-55
stockChecker¥ui¥ranking.py	56-57
stockChecker¥ui¥tabs.py	58-59